

Chapter 5

Scanning Networks

THE FOLLOWING CEH EXAM TOPICS ARE COVERED IN THIS CHAPTER:

- ✓ **Communication on protocols**
 - ✓ **Technical assessment methods**
 - ✓ **Vulnerabilities**
 - ✓ **Vulnerability scanners**
 - ✓ **Network security**
 - ✓ **Port scanning**
 - ✓ **Security testing methodology**
-

With all the reconnaissance and information gathering behind us, we can start moving into interacting with the systems in the target and its networks. This stage requires the data gathered from the reconnaissance and footprinting steps. Without that, you will have no idea what to scan. Of course, if you are working hand in glove, as it were, with the customer, they may have provided you with a lot of the details you would normally have gotten with reconnaissance techniques. Either way is OK, as long as you are clear up front about what the scope and scale of the engagement is and, once you start directly interacting with systems, you don't move beyond what was agreed to with your target.

ETHICS NOTE

Just a reminder, especially as we start moving into touching systems, that you must get permission. Even though we're just talking about scanning and not performing exploits, it's entirely possible for a scan to knock over a system. On really fragile systems, such as older embedded devices, a simple port scan can cause the device to fail. Ensure that you have permission from your target/client and that they have an understanding of what may happen once you get started. Expect the unexpected and inform your client/employer.

A common step to move to when first interacting with target systems is to perform a port scan. A port scan identifies open ports on systems connected to the target network. A port scan isn't done just for the purpose of doing a port scan. A port scan is a starting point for identifying services

and applications that are listening on those ports. Keep in mind that the objective is always to identify issues on your target network so your client/employer can improve their security posture. Identifying applications can help us identify vulnerabilities that need to be addressed.

Just identifying services and applications may not provide you with a lot of information, and what information it does provide can create a lot of work for you. Knowing the application name and even the version means you need to start digging into potential vulnerabilities that may exist with that application and service. This is why we use vulnerability scanners. The vulnerability scanner can help save time during that process. You shouldn't assume, though, that the vulnerability scanner is infallible. Vulnerability scanners can make mistakes—on both ends of the spectrum, meaning they can miss vulnerabilities as well as report ones that don't really exist. The knowledge and skill of an ethical hacker is important here—knowing how to verify and knowing what is real and what isn't.

Firewalls can be a real nuisance, as can intrusion detection (or protection) systems for that matter, if you are doing a true red team test where the operations team doesn't know what you are doing and you don't want them to know. This means that these technologies can inhibit or deter attempts to reach into the network to gather information that you can use later. Ports may be open on systems but closed in the firewall. This limits your ability to get to those applications. Your scan attempts may be detected, alerting security and operations staff about what you are doing, which may mean you just get barred altogether. Depending on the rules of the engagement, you may not want to be detected. This means there may be evasion techniques that need to be employed to make sure you can keep going and don't get blocked.

One way to achieve that, and also to poke at some functionality to identify possible vulnerabilities, is to use packet crafting. Packet crafting means you bypass the operating system and its mechanisms for creating all the data structures in the right way for transmission over the network. The reason for doing this is to create those messages incorrectly. If they don't look right, they may get ignored by firewalls or detection systems. This may allow the message to get to the endpoint.

MITRE identifies active scanning as a technique attackers use and refers to two subtechniques: scanning IP blocks and vulnerability scanning. While MITRE is correct, the description misses a lot of detail. As you will see in the rest of this chapter, scanning IP blocks and vulnerability scanning have a lot of different tools and tactics.

Ping Sweeps

Rather than blindly throwing attacks at address spaces you've identified, you may want to identify systems that are responsive within those address spaces. Responsive means that when network messages are sent to them, they provide an appropriate response to the messages. This means you can identify systems that are alive before you start aiming attacks or

probes at them. One way of determining systems that are alive is to perform a ping sweep. A ping sweep is when you send ping messages to every system on the network. The ping is an ICMP echo request, which is a common message to be sent. As long as you aren't pounding targets with an unusual number or size of these messages, they may not be noticed. Ping sweeps aren't guaranteed to succeed because there may be firewall rules that block ICMP messages from outside the network.

Using *fping*

While there are many tools that can perform a ping sweep, one of the common ones is `fping`. This is a tool designed to send ICMP echo requests to multiple systems. In the following code listing, you can see the use of `fping` to sweep my local network. The parameters used with `fping` are `-aeg`, which means `fping` shows hosts that are alive, shows elapsed time, and generates a list of targets from an address block. You'll see the list of hosts that respond at the front of the list. After that, you start to see host unreachable messages, indicating that the host is down. `fping` will send multiple messages to systems before giving up and determining it's down. This output is truncated due to length, but without the `-a`, the end of the list would be all of the hosts that were flagged as down. As it is, all we get is the indication that the systems were up.

`fping` Output

```
$ fping -aeg 192.168.86.0/24
192.168.86.1 (10.3 ms)
192.168.86.2 (16.4 ms)
192.168.86.12 (27.7 ms)
192.168.86.21 (17.4 ms)
192.168.86.11 (173 ms)
192.168.86.20 (82.7 ms)
192.168.86.31 (0.04 ms)
192.168.86.30 (14.3 ms)
192.168.86.32 (16.4 ms)
192.168.86.35 (16.9 ms)
192.168.86.37 (21.7 ms)
192.168.86.38 (20.4 ms)
192.168.86.39 (22.2 ms)
192.168.86.22 (216 ms)
192.168.86.43 (15.6 ms)
192.168.86.44 (14.7 ms)
192.168.86.49 (0.37 ms)
192.168.86.50 (14.3 ms)
192.168.86.51 (18.8 ms)
192.168.86.52 (15.3 ms)
192.168.86.28 (294 ms)
192.168.86.58 (19.9 ms)
192.168.86.47 (375 ms)
192.168.86.53 (508 ms)
192.168.86.63 (404 ms)
192.168.86.160 (15.6 ms)
192.168.86.162 (25.7 ms)
192.168.86.170 (14.6 ms)
192.168.86.189 (18.9 ms)
192.168.86.196 (25.6 ms)
```

```
192.168.86.200 (32.7 ms)
192.168.86.205 (72.2 ms)
192.168.86.210 (18.3 ms)
192.168.86.245 (15.6 ms)
192.168.86.247 (23.4 ms)
192.168.86.250 (25.9 ms)
ICMP Host Unreachable from 192.168.86.31 for ICMP Echo sent to 192.168.86.4
ICMP Host Unreachable from 192.168.86.31 for ICMP Echo sent to 192.168.86.4
ICMP Host Unreachable from 192.168.86.31 for ICMP Echo sent to 192.168.86.3
ICMP Host Unreachable from 192.168.86.31 for ICMP Echo sent to 192.168.86.3
ICMP Host Unreachable from 192.168.86.31 for ICMP Echo sent to 192.168.86.7
ICMP Host Unreachable from 192.168.86.31 for ICMP Echo sent to 192.168.86.7
ICMP Host Unreachable from 192.168.86.31 for ICMP Echo sent to 192.168.86.6
ICMP Host Unreachable from 192.168.86.31 for ICMP Echo sent to 192.168.86.6
```

Since the `-e` parameter was passed to `fping`, it provides the elapsed time. This is the round-trip time between the message that was sent and the response that was received. Some of these round-trip times are fairly high, considering it's all on the local network, even if `fping` is running on a virtual machine, where the host operating system is using wireless.

While we have a lot of systems indicating responses, that doesn't mean those are the only systems that exist on the network. Since this is the local network, we can be reasonably sure that all of this information is correct. However, network- and host-based firewalls may also block these ICMP echo requests. Just because you don't get a response does not mean that the hosts are not up. It may mean that there is something that's blocking the message. A firewall may respond with an ICMP host unreachable, just as you see in the preceding code, or it may simply drop the message, meaning there would be no response. There may be other reasons for lack of a response, though.

Using MegaPing

Another tool that can perform a ping sweep, as well as several other functions, is MegaPing. MegaPing is a GUI-based tool that runs on Windows. It incorporates several functions into a single interface. The ping sweep can be accomplished using the IP Scanner tool, which you would select from the list on the left side. [Figure 5.1](#) shows the full interface after running the IP scan. By default, the MAC address and hostnames are not included in the list. There are check boxes on the right side, however, that can be checked even after you have run a scan, and the information will be added.

Along the left side, you will see several other tools that you can make use of. It becomes a bit of a Swiss Army knife for the purposes of network troubleshooting. In addition to troubleshooting network issues, there is also a port scanning tool. I'll cover port scanning in more detail in the next section. There are other enumeration utilities you may find useful as you are identifying systems and services on your target network.

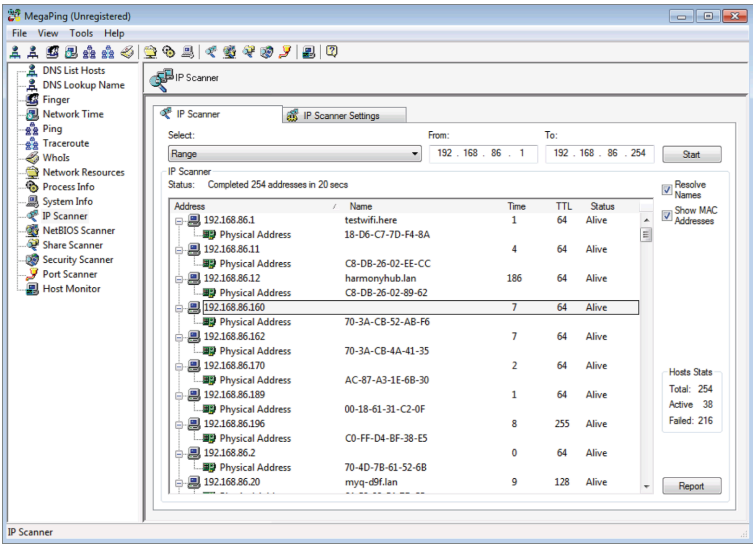


FIGURE 5.1 MegaPing IP Scanner

HANDS-ON ACTIVITY

Download a copy of MegaPing from the Magento Software website. Run a ping sweep using the IP scanner. Make sure to identify the MAC addresses for all the systems.

The ping sweep may be telling, but it's only a start. It may be used just to get a sense of the number of hosts that you may need to think about testing. The ping sweep will give you only a limited amount of information. Just knowing a host is up (and the information that a host isn't up may be unreliable) doesn't tell you a lot. Ultimately, you want to know what services are running on the host. A ping sweep will give you an idea of what systems may be targetable. Depending on how you are approaching your testing and where you are, this may be valuable. You may also consider just moving directly to a port scan, especially since port scanners will often do a ping ahead of the port scan to make sure the system is up before bothering to send port probes.

Port Scanning

Let's clear this up, in case there is confusion. We aren't talking about passing by a cruise ship, peering in windows. We are talking about network communication devices. A port is a construct within the operating system's network stack. When an application has network service functionality, it binds to a port, meaning it reserves the port and registers the application to get messages that come in on that port. Any communication received by the system addressed to one of the ports gets forwarded to the application that is registered to that port. When there is an application listening on a port, it is considered to be open. Remember that ports exist at the Transport layer, so applications determine whether they are going to use UDP or TCP as the protocol to listen on. The reason for mentioning this is that the objective of port scanning is to identify the software that is bound to the ports that are identified as open.

TCP, as you should know, uses a three-way handshake to initiate connections. To accomplish the handshake, TCP makes use of flag settings, which means there is a set of bits that are enabled or disabled to set or unset the flags. The three-way handshake uses the SYN and ACK flags to complete the connection process. Other flags, such as URG, PSH, and FIN, are used for other purposes, and the RST flag is used to let other systems know to cease communications on the destination port in the received message. Port scanners make use of the known rules in the protocol to make determinations about whether a port is open or not.

Open ports should respond to a SYN message with a SYN/ACK. Closed ports should respond to a SYN message with a RST message. What happens, though, if we send other messages to open or closed ports? Why would we even do that, considering that we know how open and closed ports respond? The reason relates to security technologies like firewalls and intrusion detection systems. Other TCP messages are used to bypass these devices and get responses where there otherwise may not be any response. The way the protocol is expected to work is documented and so network stack implementations adhere to the documentation. There are some exchanges that are simply not documented, so behavior isn't guaranteed because it's not expected. These exchanges can be used to elicit responses from our target systems. UDP is another story altogether. There is no defined way of beginning a conversation from the standpoint of the protocol. UDP messages are sent from a client to a server, and it's up to the server how it responds. The operating system's network stack has no role other than to pass the message up to the application once the Transport layer headers have been processed. This can be a challenge for port scanners. The reason is that with no defined response, it's hard to determine whether a lack of response is because of a closed port or just because the application didn't receive what it expected. It could also be that the server application listening at that port simply doesn't respond.

Since there is no defined response, port scanners have to make a best guess. If there is no response to a probe message, port scanners don't assume the port is closed because it may not be. Not only may the application just not have responded, but it's possible the UDP message was lost in transmission, since nothing in the protocol ensures that it gets from end to end. Because either is possible, the probe messages will be re-sent. There is a delay of some small period of time between each message. Sending multiple messages with delays between them can cause significant UDP port scans to take quite a bit more time than a TCP scan.

As noted earlier, port scanners may send ICMP echo requests to targets before running a port scan. There isn't much point in sending thousands of messages to a host that isn't there. This behavior can be controlled, depending on the port scanner being used.

nmap

The de facto port scanner is `nmap`, short for network mapper. After all, if it's good enough for Carrie-Anne Moss in *The Matrix Reloaded* and Rihanna in *Ocean's 8*, it should be good enough for me or you. This is a program that has been around since 1997 and has become so commonly

used that other port scanners implement the same command-line parameters because they are so well-known. It isn't just a port scanner, though; its primary role and other functions are just extensions of the core purpose of `nmap`.

`nmap` can perform UDP scans as well as multiple types of TCP scans when it comes to port scanning. In addition, `nmap` will detect operating system types, applications, and application versions. Perhaps more significantly, `nmap` supports running scripts. These scripts allow anyone to extend `nmap`'s functionality. The scripting engine, powered by the Lua programming language, has modules that scripts can be built on top of to make the job of probing systems much easier.

ADMINISTRATIVE PRIVILEGES

Many of the types of port scanning require administrative privileges because the scanner is trying to get the operating system to send packets that wouldn't normally be sent. This requires creating packets from scratch rather than just using the normal network interface to the operating system. This requires administrative privileges, which is why you will see the use of the utility `sudo` in the following examples. This grants temporary administrative privileges.

TCP Scanning

As it's the most detailed and complex type of scanning done, I'll cover the different types of TCP scans that `nmap` can perform. First, we know that transport protocols use 2 bytes for the port number in their headers. This means there are 65,536 possible ports (0–65535). Scanning that many ports, especially considering that the vast majority of them aren't used by listening applications, is very time-consuming. To be efficient, `nmap` will scan only 1,000 ports by default, though you can specify any ports for `nmap` to scan that you would like. These 1,000 ports are the ones that are most likely to have a listening service.

There are many types of TCP scanning. One of the first ones to look at is the SYN scan. This is sometimes called a *half-open scan*, because connections are left half open. `nmap` will send a SYN message to the target. If the port is open, it responds with a SYN/ACK message, and `nmap` will respond to that with a RST message, indicating it doesn't want to continue with the connection. If the port is closed, the target system will respond with its own RST message. In the following code listing, you can see a SYN scan, which is called with `-sS` as the parameter. Then, you need a target. This scan is a single IP address, but you can also specify a range or a network block.

SYN Scan with `nmap`

```
$ sudo nmap -sS 192.168.86.32
Starting Nmap 7.92 ( https://nmap.org ) at 2022-11-15 17:46 EST
Nmap scan report for billthecat.lan (192.168.86.32)
```

```

Host is up (0.022s latency).
Not shown: 500 closed ports, 495 filtered ports
PORT      STATE SERVICE
22/tcp    open  ssh
88/tcp    open  kerberos-sec
445/tcp    open  microsoft-ds
548/tcp    open  afp
5900/tcp   open  vnc
MAC Address: AC:87:A3:36:D6:AA (Apple)

nmap done: 1 IP address (1 host up) scanned in 3.69 seconds

```

To demonstrate the use of a network block as a target, we can make use of a full connect scan. Rather than a RST as the response to the SYN/ACK, `nmap` will complete the connection and then tear it down once the connection is complete. What you will see in the following code listing is the use of a CIDR block as the target address. This means `nmap` will scan the entire subnet. You will also notice that the ports are specified. Rather than just defaulting to the 1,000 ports `nmap` will usually scan, we're only going to get hosts that have port 80 and 443 open. There could be many hosts on the network that won't respond to a scan like this.

nmap Full Connect Scan

```

$ nmap -sT -p 80,443 192.168.86.0/24
Starting Nmap 7.92 ( https://nmap.org ) at 2022-11-15 17:46 EST
RTTVAR has grown to over 2.3 seconds, decreasing to 2.0
Nmap scan report for testwifi.here (192.168.86.1)
Host is up (0.011s latency).

PORT      STATE SERVICE
80/tcp    open  http
443/tcp    closed https
MAC Address: 18:D6:C7:7D:F4:8A (Tp-link Technologies)

Nmap scan report for 192.168.86.2
Host is up (0.011s latency).

PORT      STATE SERVICE
80/tcp    closed http
443/tcp    closed https
MAC Address: 68:05:CA:46:70:88 (Intel Corporate)

Nmap scan report for 192.168.86.11
Host is up (0.022s latency).

PORT      STATE SERVICE
80/tcp    closed http
443/tcp    closed https
MAC Address: C8:DB:26:02:EE:CC (Logitech)

Nmap scan report for harmonyhub.lan (192.168.86.12)
Host is up (0.014s latency).

PORT      STATE SERVICE
80/tcp    closed http

```

```

443/tcp closed https
MAC Address: C8:DB:26:02:89:62 (Logitech)

Nmap scan report for myq-d9f.lan (192.168.86.20)
Host is up (0.026s latency).

PORT      STATE SERVICE
80/tcp    open  http
443/tcp   closed https
MAC Address: 64:52:99:54:7F:C5 (The Chamberlain Group)

```

There are two things you may note in the output. First, we don't know what the application is. All we know is the protocol that is being used. Of course, we assume it's a web server, but which one? Even if we knew what the operating system is, we can't assume which application is being used as the web server. The second thing to notice is that along with the MAC address, you get the vendor. There is nothing special here. `nmap` looks up the organizationally unique identifier (OUI) part of the MAC address from a database and presents the result of the lookup.

There are additional TCP scans that `nmap` can run. The other scans make use of unexpected input as a way of potentially getting different responses. For example, the following code listing is something referred to as an Xmas scan. The reason for that is that the packets being sent have the FIN, PSF, and URG flags set, which makes the packet look lit up like a Christmas (or Xmas) tree. What you will likely notice quickly is that we don't get an indication about open ports here, as we did in the preceding code. Instead, `nmap` is telling us that the port is either open or filtered. The reason for this is that with ports that are closed, the system responds with a RST. Ports that are open don't respond at all because this is not a legal packet from the perspective of the protocol. If `nmap` doesn't get any response, it's not clear whether it's because a network device dropped the message or if the system just didn't respond to an illegal message.

Running an Xmas Scan with Nmap

```

$ sudo nmap -sX 192.168.86.32
Starting Nmap 7.92 ( https://nmap.org ) at 2022-11-15 17:46 EST
Nmap scan report for billthecat.lan (192.168.86.32)
Host is up (0.0076s latency).
Not shown: 995 closed ports
PORT      STATE SERVICE
22/tcp    open|filtered ssh
88/tcp    open|filtered kerberos-sec
445/tcp    open|filtered microsoft-ds
548/tcp    open|filtered afp
5900/tcp   open|filtered vnc
MAC Address: AC:87:A3:36:D6:AA (Apple)

Nmap done: 1 IP address (1 host up) scanned in 13.13 seconds

```

The same ports show up here as with the SYN scan earlier. The only difference is that `nmap` can't specifically determine whether the port is open or filtered. Other scans, such as the Null scan where no flags are set, will also show results as being open or filtered for the same reason. The FIN scan also uses an unexpected set of flags since the FIN flag should be sent only in cases where there is an established connection. You will also get open or filtered from the FIN scan.

UDP Scanning

UDP scanning is much more straightforward than TCP scanning. There are no options for UDP scanning. `nmap` sends out UDP messages and then watches whatever responses may come back. The expectation is that if a port is closed, the system will respond with an ICMP port unreachable message. If a port is open, the service may respond with something or it may just not respond at all. In the following code, you can see a UDP scan run with `nmap`. You'll notice on the command line that a new parameter has been added, `-T 4`. This sets the throttle rate. By default, the throttle is set at 3, which is a common rate of message transmission. If you want it faster, you can go up to 5. If you want it to go slower, potentially to avoid detection, you can turn it down to 1. Since this is on my local network and I don't care how fast it transmits, I have the throttle rate set for 4.

```
$ nmap -sU -T 4 192.168.86.32
Starting Nmap 7.92 ( https://nmap.org ) at 2022-11-15 17:46 EST
Nmap scan report for billthecat.lan (192.168.86.32)
Host is up (0.0053s latency).
Not shown: 750 closed ports, 247 open|filtered ports
PORT      STATE SERVICE
123/udp    open  ntp
137/udp    open  netbios-ns
5353/udp   open  zeroconf
MAC Address: AC:87:A3:36:D6:AA (Apple)

Nmap done: 1 IP address (1 host up) scanned in 5.21 seconds
```

[Figure 5.2](#) will show you what these requests look like after capturing them in Wireshark, a packet capturing program. At the top of the list of packets are the probe requests from the system running `nmap`. If you look at the packet decode in the bottom pane, you will see there is no data in the packet. The UDP header is the 8 bytes expected for a UDP header and then there is no payload. The bottom of the list of packets shows the ICMP port unreachable messages from the target host.

You can compare the time that it takes to perform the different scans. The SYN scan took a bit over three seconds, while the UDP scan took just over five seconds. You'll also notice that there are 1,000 ports scanned for UDP as well. We still have the problem, though, of not knowing for sure what applications are running behind these ports. This is also something `nmap` can take care of for us.

PORT SCANNING

Download a copy of `nmap`. You can use the command-line version or the GUI version with Zenmap. Even if you have only a single computer, you should have a router on your network as well. If you have nothing else, you can scan the IP address that belongs to your router, also known as the *default gateway*. Try running both TCP scans as well as UDP scans. Try two different types of TCP scans to see if you get any different results.

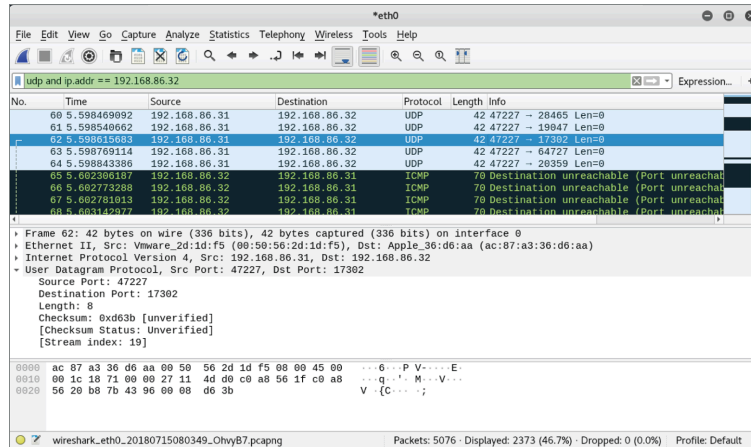


FIGURE 5.2 UDP scan from Wireshark

Detailed Information

We can use `nmap` to address the problem of not knowing the application. We can use version scanning. What `nmap` does when we run a version scan (`-sV`) is connect to the port and, as necessary, issue the correct protocol commands to get the application banner back. The banner is protocol-specific and may include such information as the software name and version. In the following code listing, you can see not only the protocol, as you've seen before, but also the software being used and the version number. For port 22, the system is running OpenSSH version 7.4. The 2.0 in parentheses indicates it is version 2.0 of the protocol. This is something that the service indicates in the banner.

Nmap Version Scan

```
$ nmap -sV 192.168.86.32
Starting Nmap 7.92 ( https://nmap.org ) at 2022-11-15 17:46 EST
Nmap scan report for billthecat.lan (192.168.86.32)
Host is up (0.0083s latency).
Not shown: 995 closed ports
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 7.4 (protocol 2.0)
88/tcp    open  kerberos-sec Heimdal Kerberos (server time: 2018-07-15 02:51:39Z)
445/tcp    open  microsoft-ds?
548/tcp    open  afp          Apple AFP (name: billthecat; protocol 3.4; OS X 10.9 - 10.11)
5900/tcp   open  vnc          Apple remote desktop vnc
MAC Address: AC:87:A3:36:D6:AA (Apple)
Service Info: OSs: OS X, Mac OS X; CPE: cpe:/o:apple:mac_os_x:10.9, cpe:/o:apple:mac_os_x
```



```
Service detection performed. Please report any incorrect results at https://nmap.org/subn
Nmap done: 1 IP address (1 host up) scanned in 35.23 seconds
```

`nmap` knows the details about the services because the application provides the information when `nmap` connects. Different applications and protocols will provide different sets of information, which means `nmap` has to understand how to speak these protocols to get this information in some cases. As an example, with web servers such as Apache, the system administrator is in charge of how much should be provided in the headers going back to the client. Apache can provide not only the name of the product, Apache, but also the version of the software and the modules that are loaded as well as the versions of the modules.

You may also want to know what operating system is running on the remote system. This is also something `nmap` can take care of, with an operating system scan. To make a determination about the operating system, `nmap` has a database of fingerprints. The fingerprints contain details about how each operating system behaves, including how the IP identification field is generated, the initial sequence number, the initial window size, and several other details. To identify the operating system, `nmap` has to find at least one open port and one closed port.

In the following code, you can see an operating system scan. You will notice that even though I didn't indicate a TCP scan, `nmap` performed one. The same ports that were found to be open before have been identified again. You will also notice that `nmap` scanned two different systems. This is another way `nmap` can be told to scan systems rather than ranges or network blocks. If you provide multiple systems on the command line, `nmap` will perform the same scan on each of the systems specified.

Operating System Scan with Nmap

```
$ sudo nmap -O 192.168.1.144 192.168.1.214
Starting Nmap 7.93 ( https://nmap.org ) at 2023-02-05 18:46 EST
Nmap scan report for deathtongue (192.168.1.144)
Host is up (0.042s latency).
Not shown: 995 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
88/tcp    open  kerberos-sec
445/tcp   open  microsoft-ds
5000/tcp  open  upnp
7000/tcp  open  afs3-fileserver
MAC Address: 14:98:77:31:B2:33 (Apple)
No exact OS matches for host (If you know what OS is running on it, see https://nmap.org/
TCP/IP fingerprint:
OS: SCAN(V=7.93E=4%D=2/5%OT=22%CT=1%CU=35270%PV=Y%DS=1%DC=D%G=Y%M=149877%TM
OS:=63E04033%P=x86_64-pc-linux-gnu)SEQ(SP=103%GCD=1%ISR=107%TI=Z%CI=RD%II=R
OS:I%TS=21)OPS(O1=M5B4NW6NNT11SLL%O2=M5B4NW6NNT11SLL%O3=M5B4NW6NNT11%O4=M5B
OS:4NW6NNT11SLL%O5=M5B4NW6NNT11SLL%O6=M5B4NNT11SLL)WIN(W1=FFFF%W2=FFFF%W3=F
OS:FFF%W4=FFFF%W5=FFFF%W6=FFFF)ECN(R=Y%DF=Y%T=40%W=FFFF%O=M5B4NW6SLL%CC=Y%Q
OS:=)T1(R=Y%DF=Y%T=40%S=0%A=S+%F=AS%RD=0%Q=)T2(R=N)T3(R=N)T4(R=Y%DF=Y%T=40%
OS:W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T5(R=Y%DF=N%T=40%W=0%S=Z%A=S+%F=AR%O=%RD=0%Q=
OS:)T6(R=Y%DF=Y%T=40%W=0%S=A%A=Z%F=R%O=%RD=0%Q=)T7(R=Y%DF=N%T=40%W=0%S=Z%A=
```



```
OS: S%F=AR%O=%RD=0%Q= )U1( R=Y%DF=N%T=40%IPL=38%UN=0%RIPL=G%RID=G%RIPCK=G%RUCK
OS: =0%RUD=G) IE( R=Y%DFI=S%T=40%CD=S)
```

```
Network Distance: 1 hop
```

```
Nmap scan report for limekiller (192.168.1.214)
```

```
Host is up (0.053s latency).
```

```
Not shown: 997 filtered tcp ports (no-response)
```

```
PORT      STATE SERVICE
```

```
135/tcp   open  msrpc
```

```
139/tcp   open  netbios-ssn
```

```
445/tcp   open  microsoft-ds
```

```
MAC Address: 54:14:F3:B8:A3:9E (Intel Corporate)
```

```
Warning: OSScan results may be unreliable because we could not find at least 1 open and 1
```

```
Aggressive OS guesses: Microsoft Windows 10 (95%), Microsoft Windows Server 2008 SP1 (90%)
```

```
No exact OS matches for host (test conditions non-ideal).
```

```
Network Distance: 1 hop
```

```
OS detection performed. Please report any incorrect results at https://nmap.org/submit/.
```

```
Nmap done: 2 IP addresses (2 hosts up) scanned in 116.89 seconds
```

One thing to keep in mind when you do an operating system scan is that `nmap` relies on fingerprints, and unlike people, the same fingerprint can match multiple operating systems. As long as nothing in the network stack has changed, you will find that multiple versions of a commercial operating system will match the same fingerprint. You can see this for the scan of the Windows system. The version running on the target does fall into the range indicated by `nmap`. One other thing to keep in mind is that `nmap` tracks the operating system. The operating system is really the kernel—the piece of software that manages the hardware, manages memory, and manages processes. All the other stuff that helps the user interact with the operating system to do user-useful things is in the operating environment, to help draw distinctions. In the case of Linux, the same kernel may be used across multiple distributions, but since the only thing being identified is the kernel, there is no way to know what distribution. `nmap` can't tell you Ubuntu versus CentOS, for example. It only knows the version of the Linux kernel you are running. Because we rely on fingerprints for the operating system, `nmap` can't always detect the version if the operating system is newer. This is the case with the Apple system. We know it's an Apple system because of the MAC. The target is running Ventura 13.2, which clearly has not been added to the `nmap` database as of the time of this scan.

Scripting

We've seen a lot of the functionality that `nmap` has. Even with all that functionality, we can go beyond. `nmap` includes a scripting engine, which allows you, as an `nmap` user, to extend the functionality in any way that you would like. It's not entirely about extending the functionality yourself, though. The scripting engine is there, but it's not completely up to you to determine what you want to do with it. There are hundreds of scripts available with the latest version of `nmap`, and the number continues to grow. They are grouped into categories, which currently are auth, broadcast, brute, default, discovery, dos, exploit, external, fuzzer, intru-

sive, malware, safe, version, and vuln. You can have `nmap` run all the scripts from a particular category. The following code is `nmap` being asked to run all the scripts in the discovery category.

Nmap Discovery Scripts

```
$ sudo nmap -sS --script=discovery 192.168.86.0/24
Starting Nmap 7.92 ( https://nmap.org ) at 2022-11-15 17:48 EST
Pre-scan script results:
|_ http-robtext-shared-ns: *TEMPORARILY DISABLED* due to changes in Robtext's API. See https
| targets-asn:
|_ targets-asn.asn is a mandatory parameter
|_ hostmap-robtext: *TEMPORARILY DISABLED* due to changes in Robtext's API. See https://www.
| ipv6-multicast-mld-list:
|   fe80::a483:e7ff:fe9c:6f65:
|     device: ens33
|     mac: a6:83:e7:9c:6f:65
|     multicast_ips:
|_       ff02::fb (mDNSv6)
| targets-ipv6-multicast-mld:
|   IP: fe80::a483:e7ff:fe9c:6f65 MAC: a6:83:e7:9c:6f:65 IFACE: ens33
|
|_ Use --script-args=newtargets to add the results as targets
| broadcast-ping:
|   IP: 192.168.79.2 MAC: 00:50:56:f6:4b:60
|   IP: 192.168.79.1 MAC: a6:83:e7:9c:6f:65
|_ Use --script-args=newtargets to add the results as targets
| targets-ipv6-multicast-echo:
|   IP: fe80::a483:e7ff:fe9c:6f65 MAC: a6:83:e7:9c:6f:65 IFACE: ens33
|_ Use --script-args=newtargets to add the results as targets
| targets-ipv6-multicast-invalid-dst:
|   IP: fe80::a483:e7ff:fe9c:6f65 MAC: a6:83:e7:9c:6f:65 IFACE: ens33
|_ Use --script-args=newtargets to add the results as targets
```

To use a script, you have to pass the `--script=` parameter followed by the name of the script you want to run. In addition to using the name of the script, you can indicate the category, as I did in the preceding code. On a Linux system, you will probably find all the installed scripts in `/usr/share/nmap/scripts`. On a Windows system, you will find the scripts in the Program Files directory where `nmap` is installed. You'll notice that the file extension for these scripts is `.nse` for “`nmap` scripting engine.” Scripts are written in the Lua language, and each file can be opened and read, possibly to get details about the function of the script. If you'd rather use `nmap` to get information about the script, you can use `--script-help`, passing in the name of the script. As an example, let's say you want to get the details about `http-waf-detect.nse`. Here you can see how to call `nmap` to get help and the start of the response from `nmap`.

Nmap Script Help

```
$ nmap --script-help=http-waf-detect.nse
Starting Nmap 7.92 ( https://nmap.org ) at 2022-11-15 17:51 EST
```

```
http-waf-detect
```

```
Categories: discovery intrusive
```

```
https://nmap.org/nsedoc/scripts/http-waf-detect.html
```

Attempts to determine whether a web server is protected by an IPS (Intrusion Prevention System), IDS (Intrusion Detection System) or WAF (Web Application Firewall) by probing the web server with malicious payloads and detecting changes in the response code and body.

To do this the script will send a "good" request and record the response, afterwards it will match this response against new requests containing malicious payloads. In theory, web applications shouldn't react to malicious requests because we are storing the payloads in a variable that is not used by the script/file and only WAF/IDS/IPS should react to it. If aggro mode is set, the script will try all attack vectors (More noisy)

This script can detect numerous IDS, IPS, and WAF products since they often protect web applications in the same way. But it won't detect products which don't alter the http traffic. Results can vary based on product configuration, but this script has been tested to work against various configurations of the following products:

- * Apache ModSecurity
- * Barracuda Web Application Firewall
- * PHPIDS
- * dotDefender
- * Imperva Web Application Firewall
- * Blue Coat SG 400

Another way of finding this information would be to just go to the script itself. When you are writing `nmap` scripts, there are variables that get set, and one of those is the description. This is the variable that gets printed when `script-help` is called. Additionally, you'll notice there is a section on usage, indicating how the script should be called from `nmap`. You'll also notice the `require` statements at the top. This is where the functionality from the `nmap` scripting engine is pulled in. These modules are needed for the script to be called from `nmap`.

Top of the `http-waf-detect.nse` File

```
local http = require "http"
local http = require "http"
local shortport = require "shortport"
local stdnse = require "stdnse"
local string = require "string"
local table = require "table"
```

```
description = [[
Attempts to determine whether a web server is protected by an IPS (Intrusion
Prevention System), IDS (Intrusion Detection System) or WAF (Web Application
Firewall) by probing the web server with malicious payloads and detecting
changes in the response code and body.
```

```
To do this the script will send a "good" request and record the response,
afterwards it will match this response against new requests containing
malicious payloads. In theory, web applications shouldn't react to malicious
```

requests because we are storing the payloads in a variable that is not used by the script/file and only WAF/IDS/IPS should react to it. If aggro mode is set, the script will try all attack vectors (More noisy)

This script can detect numerous IDS, IPS, and WAF products since they often protect web applications in the same way. But it won't detect products which don't alter the http traffic. Results can vary based on product configuration, but this script has been tested to work against various configurations of the following products:

```
* Apache ModSecurity
* Barracuda Web Application Firewall
* PHPIDS
* dotDefender
* Imperva Web Application Firewall
* Blue Coat SG 400

]]

---
-- @usage
-- nmap -p80 --script http-waf-detect <host>
-- nmap -p80 --script http-waf-detect --script-args="http-waf-detect.aggro,http-waf-detect
--
-- @output
-- PORT      STATE SERVICE
-- 80/tcp    open  http
-- |_http-waf-detect: IDS/IPS/WAF detected
--
-- @args http-waf-detect.uri Target URI. Use a path that does not redirect to a
--                               different page
-- @args http-waf-detect.aggro If aggro mode is set, the script will try all
--                               attack vectors to trigger the IDS/IPS/WAF
-- @args http-waf-detect.detectBodyChanges If set it also checks for changes in
--                               the document's body
```

In addition to calling individual scripts, you can have `nmap` select multiple scripts using wildcards. If, for example, you wanted to run all the scripts related to the Server Message Block (SMB) protocol version 2, you could just indicate that the scripts you want to run are named `smb2*`. This means any script that starts with `smb2` will get run. There are three that will get run if the SMB ports are found to be open. You can see calling the script and the results in the following code listing. You'll notice that the port scan identified port 445 as being open. This is the port used for the Common Internet File System (CIFS), which is an implementation of SMBv2. This is the port that triggered the running of the scripts, meaning that when `nmap` found the port to be open, it identified all the scripts that had registered that port and `nmap` ran those scripts.

Nmap Using Wildcards

```
$ sudo nmap -sS --script "smb2*" -T 4 192.168.86.32
Starting Nmap 7.92 ( https://nmap.org ) at 2022-11-15 17:54 EST
Nmap scan report for billthecat.lan (192.168.86.32)
Host is up (0.00024s latency).
Not shown: 500 closed ports, 495 filtered ports
```

```
PORT      STATE SERVICE
22/tcp    open  ssh
88/tcp    open  kerberos-sec
445/tcp    open  microsoft-ds
548/tcp    open  afp
5900/tcp   open  vnc
MAC Address: AC:87:A3:36:D6:AA (Apple)

Host script results:
| smb2-capabilities:
|   2.10:
|     Leasing
|     Multi-credit operations
|   3.00:
|     Leasing
|     Multi-credit operations
|     Encryption
|   3.02:
|     Leasing
|     Multi-credit operations
|_    Encryption
| smb2-security-mode:
|   2.10:
|_    Message signing enabled and required
|_smb2-time: Protocol negotiation failed (SMB2)

Nmap done: 1 IP address (1 host up) scanned in 3.44 seconds
```

As you can see, you can use the `nmap` scripts to collect a lot of information about different services. As of this writing, there are more than 600 scripts. Out of that 600-plus, more than 30 of them are targeted specifically at identifying if a server is potentially exposed to a vulnerability known with a Common Vulnerabilities and Exposures (CVE) identifier. Other scripts will identify systems that are vulnerable to other exposures. As another example, there is a script that looks for the Decrypting RSA with Obsolete and Weakened eNcryption (DROWN) vulnerability in servers that are running Secure Sockets Layer (SSL) version 2. This was a serious vulnerability, and it can be identified by interacting with the system using SSL.

If you have some programming experience, you may find that extending `nmap` with your own scripts is a fairly easy process. Since all the NSE scripts are in plain text, you can use any of them as a starting point or to grab code samples that you can put together to create your own script. Keep in mind that while `nmap` is a port scanner and can identify open ports, the scripting engine is used to perform deeper interactions with the application. The scripts can also make programmatic determinations based on the responses that are provided by the service. The scripting engine modules not only provide functionality to interact with the services, including registering ports with `nmap` so the script is called when the identified port is found to be open, but there are also modules for output that are presented back by `nmap` when the script is complete.

Zenmap

You can use the command line to do all your `nmap` scans, but you don't have to if you prefer to use a GUI. For years, you had to use the command line because that's all there was available. There were attempts to create GUIs to overlay on top of `nmap`, and then one year, under Google's Summer of Code project, a GUI called Zenmap was created, and it has remained the GUI version of `nmap` for years. It is, as suggested, an overlay for `nmap`. This means that what you do in Zenmap runs `nmap` underneath, and then the results from `nmap` are available in the GUI. With Zenmap, you don't have to think so much about the type of scan you are performing in the sense of the list of scan types I mentioned earlier. Instead, as you can see in [Figure 5.3](#), you select a scan by name in a pull-down menu.

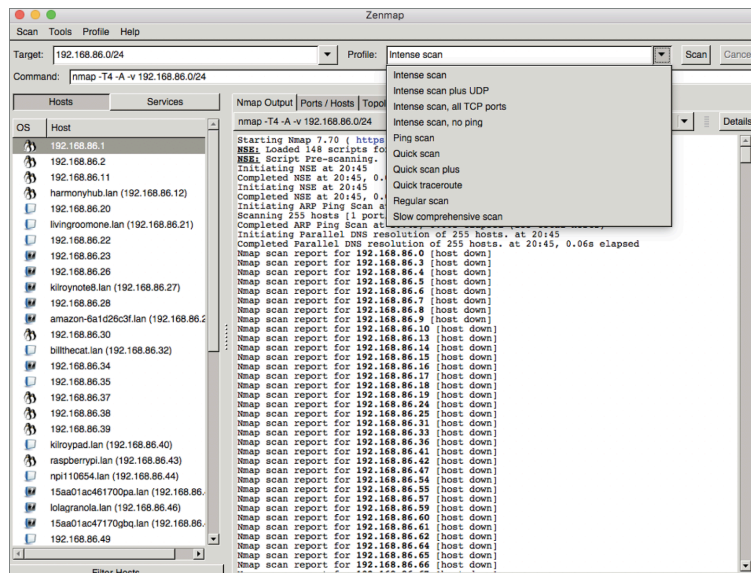


FIGURE 5.3 Zenmap scan types

You will also see the command box. Selecting the different scan types changes the command line. Instead of types like SYN scan or full connect scan, you will select from intense scan, quick scan, regular scan, and others. With the intense scans, the throttle is set high in order to complete the scan faster. A regular scan doesn't change the throttle speed. Interestingly, a slow comprehensive scan also turns up the throttle. If you don't want to use any of the ones that are provided in the interface, you can change the command line to anything you want and still run it.

The advantage to using Zenmap isn't being able to select canned scan settings but instead to visualize the output. If you use the command line, you get a lot of text output, and then you have to extract what you need from that. Using Zenmap, you can see all the hosts that were identified on the left side. You will also get a small icon indicating the operating system type, assuming you performed a scan that detected the operating system. Of course, you'll also get the regular `nmap` output if you'd rather look at that. It may be easier, though, to let Zenmap organize the results for you.

[Figure 5.4](#) shows another way of visualizing the output. Again, one of the important aspects of doing a port scan is to identify the services and, sub-

sequently, the applications. On the left side in [Figure 5.4](#), clicking the Services button shows the list of all the services that were identified in the scan. Selecting one of the services will bring up a list of all the hosts that were running that service during the scan. If you look on the right side of [Figure 5.4](#), you will see the list of hosts, but you will also see, in some cases, the application and version running on that host.

Service	Hostname	Port	Protocol	State	Version
ftp	192.168.86.1	80	tcp	open	Google WFI http admin
afs3-filer	192.168.86.2	3128	tcp	open	Proxmox Virtual Environment REST API 3.0
ajp13	192.168.86.20	80	tcp	open	
blackice-icecap	192.168.86.22	8008	tcp	open	
cslistener	192.168.86.22	9080	tcp	open	Mongoose httpd
domain	192.168.86.30	3128	tcp	open	Proxmox Virtual Environment REST API 3.0
http	192.168.86.35	80	tcp	open	Netgear ProSafe Gigabit Web Managed (Plus) swit
http-proxy	192.168.86.38	80	tcp	open	SAGE EAS Digital Endec remote audio monitor/lev
jettidirect	192.168.86.39	49152	tcp	open	DirectTV Genie
kerberos-sec	192.168.86.39	9000	tcp	open	DirectTV HMC-Lite
microsoft-ds	192.168.86.39	8083	tcp	open	Mongoose httpd
msrpc	192.168.86.39	8082	tcp	open	Mongoose httpd
ms-wbt-server	192.168.86.39	8080	tcp	open	DirectTV Set-top Box HTTP Exported Functionality
netbios-ssn	stevedallas.lan (192.168.86.50)	2869	tcp	open	Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
printer	stevedallas.lan (192.168.86.50)	5357	tcp	open	Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
radcan-http	stevedallas.lan (192.168.86.50)	10243	tcp	open	Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
rpcbind	192.168.86.52	8080	tcp	open	Apache Tomcat
rtsp	192.168.86.58	80	tcp	open	Netgear ProSafe Gigabit Web Managed (Plus) swit
rxapi	192.168.86.160	80	tcp	open	
shell	192.168.86.162	80	tcp	open	
snet-sensor-mgmt	192.168.86.245	80	tcp	open	Apache httpd
soap	192.168.86.245	443	tcp	open	Apache httpd
ssh	192.168.86.247	80	tcp	open	
tcpwrapped	192.168.86.250	80	tcp	open	

FIGURE 5.4 Zenmap service output

Another useful capability of Zenmap is its ability to save scans. OK, it goes beyond the ability to save scans. Ultimately, you could save a scan in `nmap`. What Zenmap will do is compare two saved scans. This means you can get a baseline of a network and then check it again later to see what may have changed. If you are testing the same network multiple times, being able to get the historic differences will be useful. You'll get not only hosts that are different but also services that are different.



By default, Zenmap will save scans in XML format.

Since XML is a text-based format, you could get the differences between two XML files yourself, but it's easier to have a tool that will consume XML and then compare it node by node to get more than just the text differences. Zenmap will do that for you.

While I generally prefer to use the command line, there are times when GUI tools are just far more useful. I'd prefer to run scans from the command line, but the visualization and organizational capabilities of Zenmap make it worth using. Of course, Zenmap doesn't require that you do the initial scans in Zenmap. You could run the scan in `nmap`, save the output in XML format, and then open the XML file in Zenmap and get all the goodness that we've been talking about here.

masscan

Have you ever wanted to just port scan the entire Internet to identify all the web servers that respond? Of course, if you were going to do that, you'd want to do it flat-out, as fast as you possibly can. According to `masscan`'s developer, Robert Graham, that was essentially the purpose for `masscan`. At its core, it's a port scanner. It does some of the same things that `nmap` does. The difference is that it was developed to go as fast as your system and the network connection you have will allow it to go.

Since `nmap` has become the de facto port scanner and people who are inclined to do port scans know how `nmap` works, `masscan` uses the same sorts of command-line parameters as `nmap`. Port scanning isn't really fancy, when you come down to it. You tell the port scanner what ports you want to scan and the systems you want to scan. In the following code, you can see running `masscan` to identify all the web servers. In terms of how it relates to `nmap`, you'll notice that the parameter to indicate ports is the same. You'll also notice that where I left the type of scan off, `masscan` let me know it had filled in the `-sS` for me.

masscan Identifying Web Servers

```
$ sudo masscan --rate=100000 -p80,443 192.168.86.0/24
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2022-11-15 22:55:40 GMT
Initiating SYN Stealth Scan
Scanning 256 hosts [2 ports/host]
Discovered open port 80/tcp on 192.168.86.250
Discovered open port 80/tcp on 192.168.86.1
Discovered open port 80/tcp on 192.168.86.35
Discovered open port 443/tcp on 192.168.86.44
Discovered open port 443/tcp on 192.168.86.245
Discovered open port 80/tcp on 192.168.86.247
Discovered open port 80/tcp on 192.168.86.38
Discovered open port 80/tcp on 192.168.86.44
```

The most significant difference is the addition of the `rate` parameter. This is in packets per second, and you can use fractional rates, in a decimal form, such as 0.5, to indicate a single packet every 2 seconds. The parameter provided here requests 100,000 packets per second. Your mileage will vary here, based on what your target is, how much bandwidth you have available, and how fast your system and network interface can generate and send packets to the network.

You may also have noticed that `masscan` forced the use of `--randomize-hosts`, which means that the IP addresses tested would not be in numerical order. Instead, the order will be randomized. The idea behind randomizing hosts is to potentially get around network monitoring tools. If you scan in order, it is fairly clear that a scan is happening. Randomizing scanning makes it a little less obvious what is happening. This may be especially true if you slow the rate down.

`masscan` doesn't just do port scanning, though, even really fast scanning. It can also do some information gathering, much like `nmap` can. You can request that `masscan` grab banners. This is done using the `--banners` parameter.

Getting Banners with masscan

```
$ sudo masscan -sS --banners --rate=100000 -p80,443 192.168.86.0/24
Starting masscan 1.3.2 (http://bit.ly/14GZzcT) at 2022-11-15 23:01:13 GMT
-- forced options: -sS -Pn -n --randomize-hosts -v --send-eth
Initiating SYN Stealth Scan
Scanning 256 hosts [2 ports/host]
Discovered open port 80/tcp on 192.168.86.162
Discovered open port 80/tcp on 192.168.86.1
Discovered open port 80/tcp on 192.168.86.160
Discovered open port 80/tcp on 192.168.86.250
Discovered open port 80/tcp on 192.168.86.44
Discovered open port 80/tcp on 192.168.86.35
Discovered open port 443/tcp on 192.168.86.245
Discovered open port 80/tcp on 192.168.86.196
Discovered open port 80/tcp on 192.168.86.247
Banner on port 80/tcp on 192.168.86.35: [http] HTTP/1.1 200 OK\x0d\x0aConnection: close\x0d\x0a
Banner on port 80/tcp on 192.168.86.35: [title] Redirect to Login
```

You'll notice that only one of the systems shows headers, and out of those headers, there isn't a server type. The same server suggests that there is a redirect to a login page. Running the same scan with `nmap` returns server types. `masscan` doesn't have the same capabilities as `nmap`, including a lack of support for additional scan types, such as the unexpected TCP scans like Xmas, FIN, and ACK. It also doesn't support UDP scans. However, if you want all that functionality, you can use `nmap`. It's free and more than capable for those sorts of scans. What `masscan` gets you is the ability to indicate what rate you want to scan at and the ability to perform very fast scans.

MASSCAN SCANNING

Scan the same host(s) as you did with `nmap`. Identify any differences in results. Try timing the two programs and see which one is faster.

MegaPing

We've looked at MegaPing before for its capability to perform ping sweeps. As noted earlier, MegaPing has a number of capabilities, including the ability to run port scans. These are not just run-of-the-mill scans, however. You'll remember that `nmap` scans 1,000 ports by default. These are commonly used ports. You can certainly select other ports if you want. One thing MegaPing provides us with that we don't get with `nmap` is some preselected port collections. You can see the drop-down in [Figure 5.5](#) that provides you with different selections for ports. One of these is Hostile Ports, which are ports that are commonly misused as well as ports

that may commonly be used by Trojan horse programs and other malicious software (malware). At the bottom, you will see the list of ports that are included in the scan type.

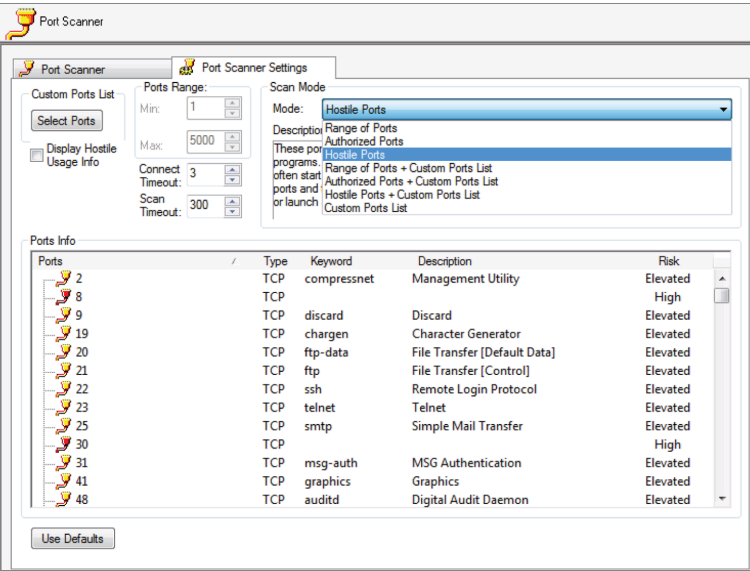


FIGURE 5.5 MegaPing scan types

To run a scan in MegaPing, you select Port Scanner on the left side of the interface. Then you need to add your targets for your scan. One downside to MegaPing is that it doesn't accept a CIDR block as a target. It also doesn't accept a range of addresses. If you want to scan multiple IP addresses, you need to add them in. You'll see the box in the middle where you can add the addresses and then select the ones you want to scan. If you just want to scan a single address, you enter it into the box below Destination Address List. Then you just click Start. Figure 5.6 shows the results of a scan of hostile ports.

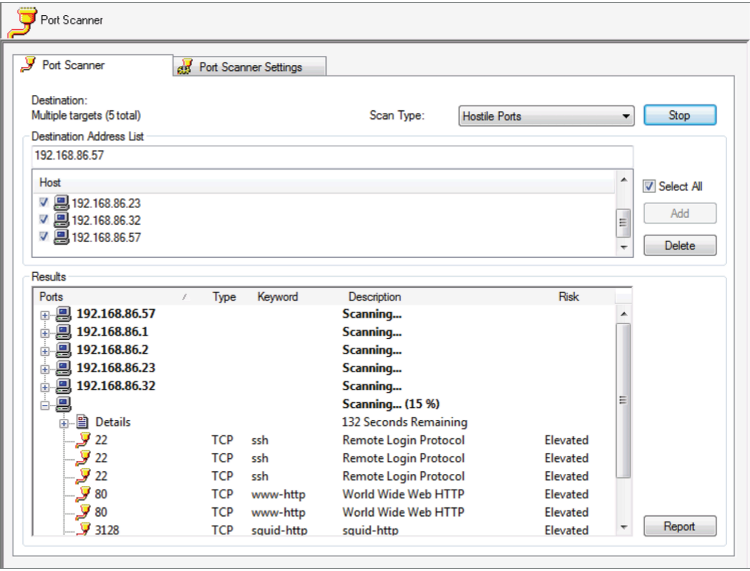


FIGURE 5.6 MegaPing scan report

One thing you may notice when you look at the results is that there are common ports listed as being open. You'll see in the note alongside ports like 80 (www-http) that they are elevated ports. These are ports that have

to be opened with administrative privileges. This means the ports will be targeted by attackers because compromising the application behind the port will immediately give the attacker administrative privileges. Another scan type is Authorized Ports, which will scan just the range of ports where applications are expected to reside. Other ports are considered to be ephemeral, meaning they are assigned to client applications as source ports when they start a conversation with a server.



One thing to note about MegaPing is that, while it has a load of functionality, it is commercial software. There is a fully functional evaluation version, but it does require you to wait while the license message shows when you start up the application.

Metasploit

While Metasploit is known primarily for being an exploit framework, which you turn to when you want to start exploiting services, meaning you want to get unauthorized access to the service, there are thousands of modules available that are not exclusively about exploiting services. We can use Metasploit for port scanning. Just to give you a sense of the types of port scanning we can do, the following is a list of the modules, at the time of this writing, available in Metasploit. This was obtained using `msfconsole`, which is one of the ways you can access Metasploit over the command line.

```
[msf](Jobs:0 Agents:0)>> search portscan

Matching Modules
=====

#  Name                                     Disclosure Date  Rank    Check  De
-  -
0  auxiliary/scanner/portscan/ftpbounce      normal         No     FT
1  auxiliary/scanner/natpmp/natpmp_portscan  normal         No     NA
2  auxiliary/scanner/sap/sap_router_portscanner  normal         No     SA
3  auxiliary/scanner/portscan/xmas           normal         No     TC
4  auxiliary/scanner/portscan/ack            normal         No     TC
5  auxiliary/scanner/portscan/tcp            normal         No     TC
6  auxiliary/scanner/portscan/syn            normal         No     TC
7  auxiliary/scanner/http/wordpress_pingback_access  normal         No     WC
```

You can see that we can use some of the same techniques we were using in `nmap`. Where `nmap` uses command-line switches, even if you are using Zenmap, Metasploit is a little more interactive. You set parameters or variables to provide the module you are using with the information needed to get the module to scan the right network in the way you want to scan it. The following is the output from a scan session using `msfconsole`:

- 1. You need to use the module, which is expressed as a path.

One reason it is expressed as a path is because it's stored as a path.

The module named `syn` is just a file named `syn.rb` in your file system, because it's a Ruby script that makes use of the library functions provided by Metasploit.

2. Once you have loaded the module, start setting parameters.

When performing a port scan, you may only need to set the remote hosts (`RHOSTS`) parameter, unless you are targeting specific ports.

```
[msf](Jobs:0 Agents:0)>> use scanner/portscan/syn
[msf](Jobs:0 Agents:0) auxiliary(scanner/portscan/syn)>> set RHOSTS 192.168.1.0/24
RHOSTS => 192.168.1.0/24
[msf](Jobs:0 Agents:0) auxiliary(scanner/portscan/syn)>> show options
```

Module options (auxiliary/scanner/portscan/syn):

Name	Current Setting	Required	Description
-----	-----	-----	-----
BATCHSIZE	256	yes	The number of hosts to scan per set
DELAY	0	yes	The delay between connections, per thread, in milliseconds
INTERFACE		no	The name of the interface
JITTER	0	yes	The delay jitter factor (maximum value by which to +/- DELAY) in milliseconds.
PORTS	1-10000	yes	Ports to scan (e.g. 22-25,80,110-900)
RHOSTS	192.168.1.0/24	yes	The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
SNAPLEN	65535	yes	The number of bytes to capture
THREADS	1	yes	The number of concurrent threads (max one per host)
TIMEOUT	500	yes	The reply read timeout in milliseconds

```
[msf](Jobs:0 Agents:0) auxiliary(scanner/portscan/syn)>> set INTERFACE ens33
INTERFACE => ens33
[msf](Jobs:0 Agents:0) auxiliary(scanner/portscan/syn)>> run
```

3. Once you have your ports selected and your hosts selected, which can be an entire block, as used here, or a list of hosts or even a single host, run the module.

Metasploit will execute the Ruby script and present the results. The following is a selection of the results:

```
[msf](Jobs:0 Agents:0) auxiliary(scanner/portscan/syn)>> run

[+] TCP OPEN 192.168.4.10:22
[+] TCP OPEN 192.168.4.15:22
[+] TCP OPEN 192.168.4.20:22
[+] TCP OPEN 192.168.4.99:22
[+] TCP OPEN 192.168.4.101:22
[+] TCP OPEN 192.168.4.136:22
[+] TCP OPEN 192.168.4.183:22
[+] TCP OPEN 192.168.4.1:53
[+] TCP OPEN 192.168.4.178:53
[+] TCP OPEN 192.168.4.196:53
[+] TCP OPEN 192.168.4.202:53
```

One advantage to running the port scan in Metasploit is that it will maintain the results in a database that you can pull back in subsequent runs of the program. You don't have to keep running the scan just because you exited `msfconsole` and started it back up again.

Of course, you can also run `nmap` from inside `msfconsole`, and Metasploit will still keep track of the results for you in the database. You'll get the same results no matter which approach you take because the rules of network protocols are universal, and if there is an application, there is an application, regardless of what program is sending a connection request to it.

Vulnerability Scanning

Knowing open ports and even applications that are listening on those ports is a good start. You can then start hunting and pecking at those ports after doing a lot of research about what vulnerabilities may exist within those applications. You could also just find a lot of exploits and start throwing them at the applications on the open ports. This may be a simple way of checking to see if there are vulnerabilities. You just check all the exploits you can find against your target. There are a few issues with that approach, however. The first is that you may end up causing failures on your target systems where you may not mean to. Blind testing can lead to unexpected results, and one of your objectives, from an ethical standpoint, is to cause no harm.

Certainly security testing of any type can lead to unexpected results and failures. However, your job is to control it as best you can and to ensure that your client or employer is aware of the possible ramifications of your testing. It wouldn't be very professional to tell your client that you're just going to throw a lot of exploits at their system without any idea what the impact would be and that they may experience outages as a result. Your job is to control your testing—to be knowledgeable about what you are doing and what the possible outcomes are.

Another issue is that if you are engaged in a red team test where the target has no idea you are running attacks, you want to be sure they don't detect you, or at least you want to do everything you can to avoid detection. Blindly running a lot of exploits against a lot of systems, including systems that may not even have the application that's vulnerable, is going to be noisy, and if there is any detection capability, you will be caught. That means you will have failed.

A better approach is to use a vulnerability scanner, which takes an intelligent approach to identifying potential vulnerabilities. A vulnerability scanner will identify open ports and listening applications and then determine what vulnerabilities may be possible based on those applications. The scanner will then run tests that have been defined for those vulnerabilities. The objective of a scanner is not to compromise a system; it is just to identify potential vulnerabilities.

This does not guarantee that what the scanner has identified is an exploitable vulnerability. It means that the scanner has found something it

believes is a vulnerability based on interactions with the target system as compared with data the vulnerability scanner has. This is called a *false positive*. Any issue found by a vulnerability scanner needs to be verified manually. This may include investigating the actual interaction as presented by the scanner—sometimes it's based on return codes without looking at the actual data, for instance. It may also involve actually redoing the test performed by the vulnerability scanner. The vulnerability scanner is a tool and shouldn't be considered to be the end of your testing. It's the starting point. In spite of how good vulnerability scanners are, they are not the terminus.



These are the four categories of vulnerabilities:

- **False Positive** The scanner has identified something it believes to be a vulnerability. After investigation, it turns out it's not really a vulnerability.
- **False Negative** The scanner has not identified a vulnerability. It later turns out that there was a vulnerability that the scanner missed.
- **True Positive** The scanner has identified a vulnerability that, after manual investigation, turns out to be a legitimate vulnerability.
- **True Negative** The scanner has not identified a vulnerability, and there is not a vulnerability to identify.

For a historical perspective, it's worth noting that network vulnerability scanners have been around since the early 1990s. The first one, developed by Dan Farmer and Wietse Venema, was known as Security Analysis Tool for Auditing Networks (SATAN). SATAN then spawned additional tools like Security Auditors Research Assistant (SARA) and Security Administrator's Integrated Network Tool (SAINT). SATAN was written primarily in Perl and used a web interface. While Perl has been replaced by other languages for modern vulnerability scanners, they do generally use web interfaces. One of the vulnerability scanners that became very popular is Nessus. We'll take a look at Nessus shortly, but we'll start with OpenVAS, which is related to Nessus.



I had tried to grab a copy of SATAN just to run it again for fun, since I last ran it 30 or so years ago. While the SATAN web page is still available, none of the mirrors that once had it are available. However, you can still get a copy of SARA, which hasn't been updated in years, and SAINT, which is now a commercial product.

OpenVAS

SATAN was open source, meaning you could look at everything SATAN was doing and, if you felt like it, extend its functionality by adding modules yourself. If you could find a copy of SATAN somewhere, you would still be able to look at the source code.

Another open source vulnerability scanner in the early 2000s was Nessus. It was initially released in 1998 as a freely available vulnerability scanner and remained so until 2005 when the company, formed three years before, closed the source code, making all future development proprietary to the company. At that point, the existing Nessus source was version 2, and the first version from Tenable was version 3. The existing source code for version 2 was open, however, and two separate projects were created, where the Nessus code for version 2, abandoned by the Nessus developers, was forked to create a basis for the new projects.

One of these forks was the Open Vulnerability Assessment System (OpenVAS). Initially, OpenVAS was the same as Nessus, as you'd expect. Over time, though, OpenVAS developed its own application architecture, using multiple tiers that Nessus hadn't explicitly used. Nessus initially had a native application client to manage the scans, and OpenVAS continued to use the same native application. OpenVAS developed the Greenbone Security Assistant (GSA) as the user interface for OpenVAS. Today, GSA is accessed through a web interface. You can see the login screen from GSA in [Figure 5.7](#).



FIGURE 5.7 Greenbone Security Assistant

OpenVAS allows you to have multiple users, each of which may have different permissions. Some users may be able to create scans, while others may only be able to look at the scan results. Other users would be able to create users and administer the OpenVAS installation. In addition to users, OpenVAS supports roles. Permissions within the roles can be altered, and new roles can be created. When you install OpenVAS, the admin user is created as part of the setup process, and a random password is generated.

Setting Up Targets in OpenVAS

A scan in OpenVAS has many components. When you create a scan, you need a target or set of targets. Creating a target also requires some infor-

mation. You need the set of IP addresses you want to run tests on. You can also exclude addresses from the set you are testing. You may do this to provide a network block in your target list, but you may also have fragile systems in that network block. Because of that, you may want to tell OpenVAS not to test that address. [Figure 5.8](#) shows the dialog box in OpenVAS where you create a target. You'll see the IP addresses as well as the exclusions.

FIGURE 5.8 Creating a target in OpenVAS

You will also see that you can create a set of ports that you want to scan. One thing a vulnerability scanner like OpenVAS does is scan ports to determine which ones it should be focusing testing on. You can determine the range of ports you want to test. This can limit the amount of testing you do if you care only about testing against particular services. This is one way of controlling the scope of what OpenVAS is doing.

You aren't always going to be doing black-box testing, meaning you aren't always going to have no information. Sometimes you will have details about your target. Those details can be useful because they can help you get a deeper sense of the vulnerabilities your target organization is subject to. For example, you may be provided with credentials, and those credentials can be used in OpenVAS. The credentials, when provided to OpenVAS, will allow the scanner to look at local vulnerabilities and not just network or remote vulnerabilities. The credentials will be used by OpenVAS to log into the system. Once OpenVAS has authenticated, it can start looking for local vulnerabilities.

While you can create credentials from the target window, you can also just create credentials from the Configuration menu. When you indicate that you want to create credentials, whether you're in the Target Create window or you are just going to Credentials from the Configuration menu, you are going to get the window shown in [Figure 5.9](#). You can create credentials that can be used across multiple protocols, using different authentication schemes. This may be just username and password, or it may be using SSH keys in place of the password. Once the credentials are created, they can be applied to your target. You can also specify a privilege elevation process. This is a separate set of credentials. This is the type

of user account that you wouldn't be able to directly log into, but you may be able to elevate to once you are logged in.

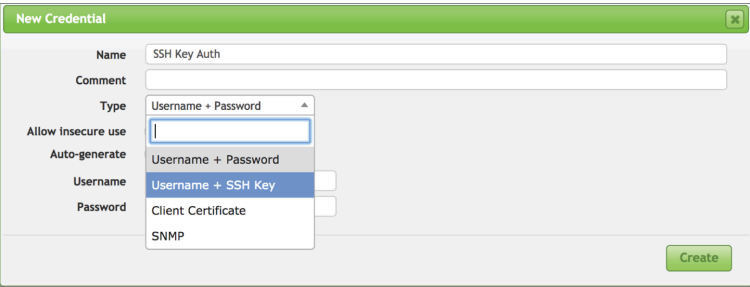


FIGURE 5.9 Creating credentials in OpenVAS

One of the issues with this is that it assumes the same credentials are used across multiple systems, since you can apply only one set of credentials per protocol. This may work if you have a centralized user account store. If not, you could also group your targets to match credentials.

Scan Configs in OpenVAS

The core of a scan is in the scan config. The scan config is the definition of what plugins are tested against the target. By default, there are eight scan configs defined in OpenVAS. You can see the list of those scan configs in Figure 5.10, as well as a scan configuration created on this system. You can see the number of network vulnerability tests (NVTs) that have been enabled in each config. The NVTs are categorized into families for organizational purposes. You can enable the entire family or just enable individual NVTs as you need to. You will notice that there is a config named Empty that has no NVTs enabled in it. Creating a new config with a minimal set of NVTs results in what you see with the MyScan config that has only three NVTs and two families.

Scan Configs (9 of 9)						
Name	Families		NVTs		Actions	
	Total	Trend	Total	Trend		
Discovery (Network Discovery scan configuration.)	22		2263			
empty (Empty and static configuration template.)	0		0			
Full and fast (Most NVTs; optimized by using previously collected information.)	62		46346			
Full and fast ultimate (Most NVTs including those that can stop services/hosts; optimized by using previously collected information.)	62		46346			
Full and very deep (Most NVTs; don't trust previously collected information; slow.)	62		46346			
Full and very deep ultimate (Most NVTs including those that can stop services/hosts; don't trust previously collected information; slow.)	62		46346			
Host Discovery (Network Host Discovery scan configuration.)	2		2			
MyScan (Empty and static configuration template.)	0		0			
System Discovery (Network System Discovery scan configuration.)	6		29			

FIGURE 5.10 OpenVAS scan configs

To create a scan config, you click the small blue icon with a star in it at the top left of the screen. You will be asked to provide a name for it and then indicate which base template you want to start with. One option is an empty config, which is what I used for MyScan, and the other is full and fast. This includes all the NVTs. So, you can build from nothing, or you can pare back from everything. You'll know how you want to think about this based on what you are doing—start from scratch and build up

or pare down from a large chunk. Once you've created the scan, you have a config you can use. Creating is as simple as naming it and determining what the base config is. You'll likely want to tune it, though, so you are running tests that are significant to your target network. This is not to say that tests will be run blindly. OpenVAS will make determinations about what tests to run based on what it finds from some initial scans. When you are ready to select different tests to run, you will need to edit your scan config. You'll make decisions about tests to run by first determining the families you want to enable. You can see a partial list of the families in [Figure 5.11](#). There are some elements here to consider. Not only can you determine whether to enable families, and which NVTs to enable, but you can also determine how a family keeps up as NVTs are added over time. You can select to keep the config static or you can have OpenVAS enable new ones as they are added to the OpenVAS installation.

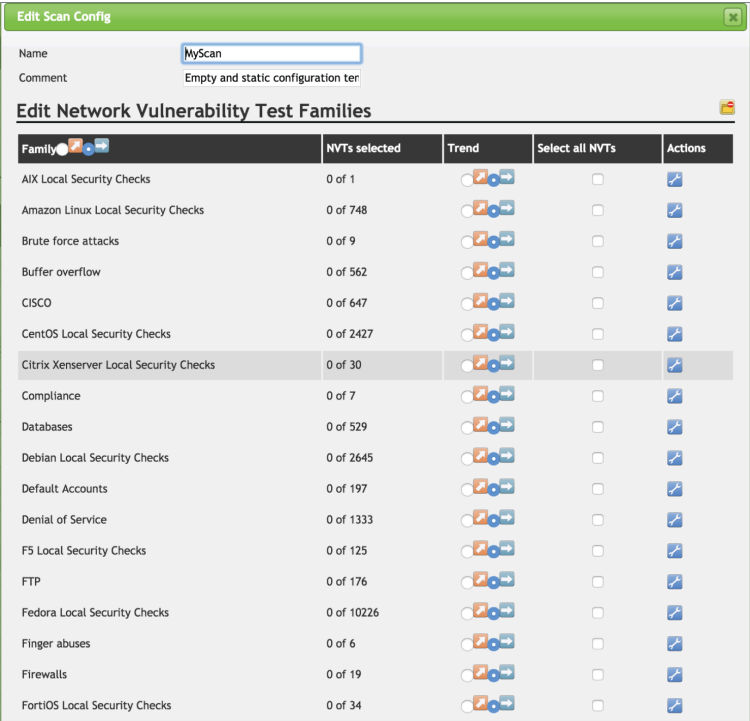


FIGURE 5.11 OpenVAS NVT families

Once you open up the family, you will get a list of all the NVTs that belong to that family. [Figure 5.12](#) shows the list of NVTs that belong to the Cisco family. In the Cisco NVTs, for instance, there are timeout values that you can change if you don't want to use the default value. This list of NVTs allows you to select exactly which ones you want to include in the config. You don't have to have large, blanket configs. You can specifically tailor your scan configs based on the environment you are testing.

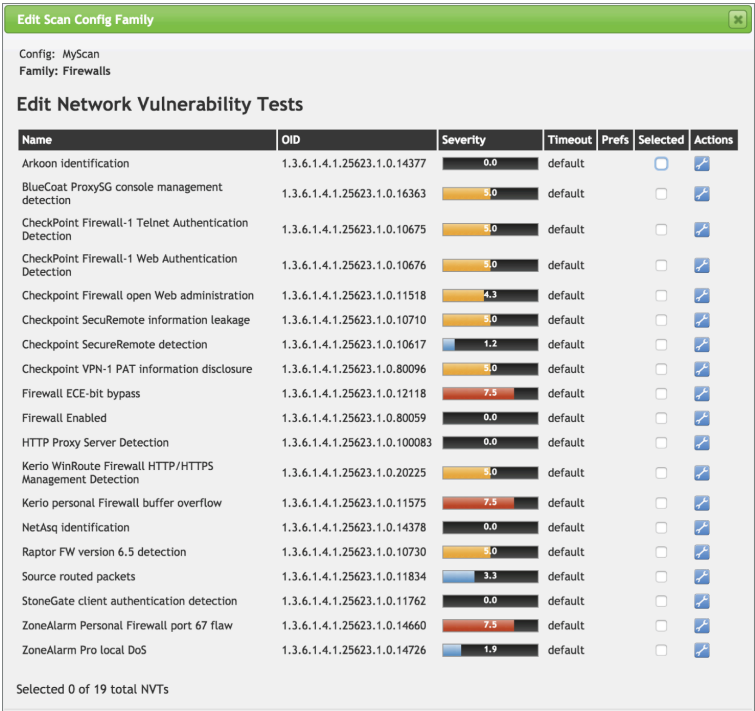


FIGURE 5.12 OpenVAS NVT selections

At this point, you have a scan config you can run against a target. This means you need to move on to creating a task that will run your scan config for you. You'll be able to do this as a one-off or in a scheduled task.

RUNNING A SCAN

One important idea to keep in mind is that once you run a scan, the focus should be identifying a remediation plan for any vulnerabilities found. Running a scan and then ignoring the results is probably worse than not running the scan at all. From a liability perspective, it means that vulnerabilities were identified, meaning they were known, without anything being done about them. At least some analysis should be performed to document a response to each vulnerability, based on a risk assessment and company policy.

Scan Tasks

Scan configs and targets are necessary to create a scan task. You'll be able to create the target as part of creating a scan task, and it will persist just as if you had gone to the target configuration separately. The scan config, though, has to be done ahead of time unless you want to use one of the prepackaged scan configs. When you go to the Scans menu and then select Tasks, you will get something that looks like what you see in Figure 5.13. This shows that there is one scan that is running. The more scans you run, the more the charts will change.

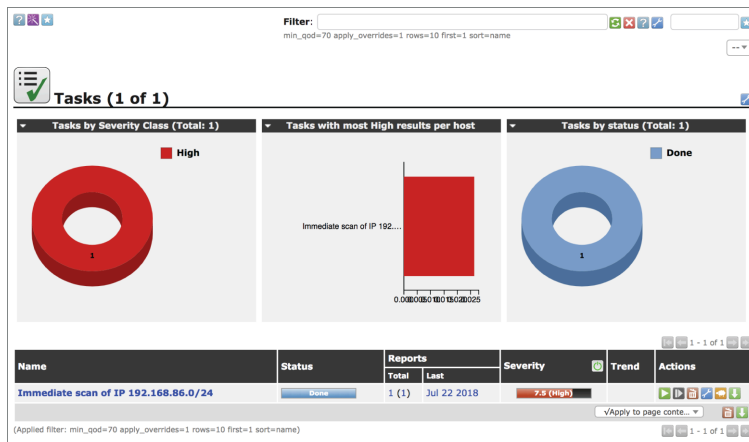


FIGURE 5.13 OpenVAS tasks

When you start a scan, you will see an icon in the upper left that looks like a sheet of paper and a star. You would hover over this and then click New Task. This will bring up the dialog box shown in [Figure 5.14](#). From here, you will need to select your target from a pull-down list. If you haven't created a target, you can click the blue star icon to create a target and save it. You can also create alerts based on severity or a filter you create. You'll be able to send alerts via email, an HTTP GET request, SMB, SNMP, or other connections. You will also need to select your scan config. This can be one of the configs you have created or one of the default configs.

The "New Task" dialog box is shown with the following fields and options:

- Name:** Regular Scan
- Comment:** (empty)
- Scan Targets:** Local Network (with a blue star icon to add new targets)
- Alerts:** (empty, with a blue star icon to add new alerts)
- Schedule:** -- (dropdown), ☐ Once
- Add results to Assets:** ☒ yes ☐ no
- Apply Overrides:** ☒ yes ☐ no
- Min QoD:** 70 %
- Alterable Task:** ☐ yes ☒ no
- Auto Delete Reports:**
 - ☒ Do not automatically delete reports
 - ☐ Automatically delete oldest reports but always keep newest 5 reports
- Scanner:** OpenVAS Default
- Scan Config:** Full and fast (dropdown menu is open showing options: Full and fast, Discovery, Full and fast ultimate, Full and very deep, Full and very deep ultimate, Host Discovery, MyScan)
- Network Source Interface:** (empty)
- Order for target hosts:** (empty)
- Maximum concurrently executed NVTs per host:** (empty)
- Maximum concurrently scanned hosts:** (empty)
- Create:** (green button)

FIGURE 5.14 OpenVAS task creation

You can create a schedule for the scan, but by default there is no schedule. The task will be created, waiting for you to start it. Using a schedule, you can have the scan run as often as you would like. You may not want it to run regularly, or you may not want to run it right away. You will be able to tell OpenVAS to just run it once, as you can see in [Figure 5.14](#). The target and scan config will be populated, assuming there is a target configured. The scan config will be populated with the first scan config in the list. These are two configuration elements you will want to make sure you

check on since they are the most important and relevant factors for your scan.

This does not mean, though, that there are no other elements that are worth looking at. If you have a large network and multiple scanner systems, you may want to select which scanner you want this to run from. You will also be able to define the source interface in case you have multiple interfaces on your scanner. You will also want to determine the order in which you want to scan your targets. This may depend on whether you are trying to hide your activities and whether you think a random selection of hosts will potentially make it look less obvious that you are running a scan. You can also help with that by reducing the number of simultaneously scanned hosts as well as the maximum number of concurrent NVTs tested against a single host.

To start up the scan, you will need to look at your list of tasks and click the green arrow that looks like a start button on an audio or video player. The scan will not run until you have started it. Just creating the scan means you have set the parameters for it. It doesn't mean you have started the scan. This is where the schedule can be useful, because OpenVAS will start the scan for you when you say rather than expecting you to start it.

Scan Results

You will be able to monitor your scan results as the scan is running. However, you will also be able to review all the completed scans and review historic vulnerabilities. The Scans dashboard will provide you with charts for visualization of your vulnerabilities over time so you can get a sense of whether the security posture of your target is improving. You can get a look at the dashboard in [Figure 5.15](#). What you see is some charts that focus on the reports. What you don't see from these charts is what the vulnerabilities from all your scans look like. At the bottom of this figure you can see a synopsis of the two scans that have been run in this installation. The immediate scan was done using the Task Wizard, which is a quick-start way of kicking off a scan.

The Scans menu provides access to different ways of looking at the results. The first way we will look at is via the Results page. This has a list of all the results from all the scans. You will also get more charts. You can see the list of results and the accompanying charts in [Figure 5.16](#). The charts are interesting, but the list of results has some information that is of note. In the first column, you will see a short name for the vulnerability. This should provide you with enough information so you will know essentially what the vulnerability is. At a minimum, you will know something about the service or device from the vulnerability name.

You will also get additional useful information. You will see that beyond the summary is the severity. The severity values would include High, Medium, and Low. To the left of the severity is the solution type. What you see in the samples here are all vendor fixes. You may also see mitigation, which means there are ways to alleviate the impact if the vulnerability were to be exploited. You may also see that there are no fixes avail-

able. While all that is shown in [Figure 5.16](#) are issues that have vendor fixes, there are also issues found that have no fixes as well as issues that have mitigations.

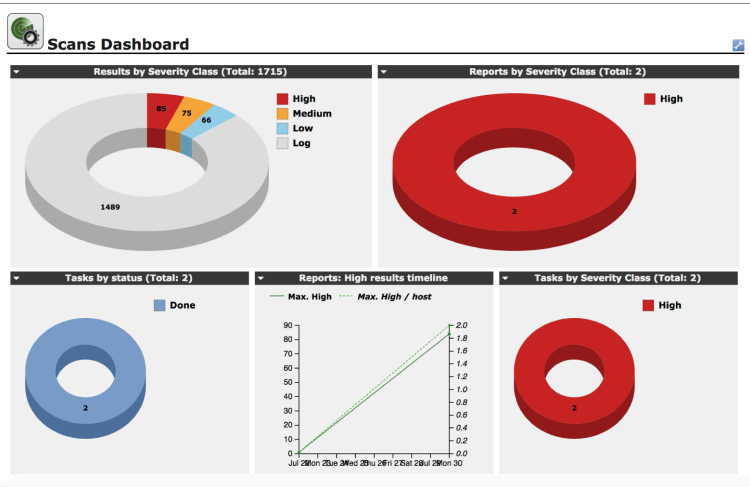


FIGURE 5.15 OpenVAS Scans dashboard

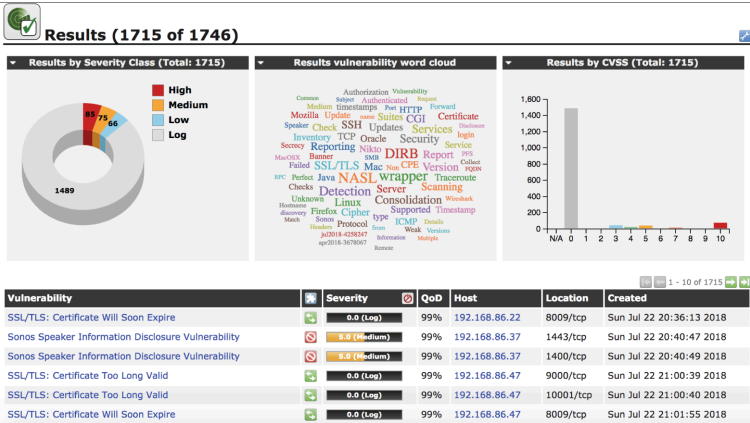


FIGURE 5.16 OpenVAS Results list

To the right of the severity is something shown as QoD. This is the quality of detection, and what it means is how certain OpenVAS is about whether the vulnerability is a true positive. You will see some very high numbers in that column for some apparent macOS vulnerabilities. Considering that the systems identified are running the latest macOS and were fully patched at the time of the scan, these high-confidence numbers are misleading. To understand exactly what OpenVAS was looking for to make the determination would require looking at the NASL file associated with the identified vulnerability.

In the far-right column, you can access actions, as shown in [Figure 5.17](#). Since these reports remain stored in OpenVAS as long as the instance remains, you probably want to make necessary changes to the findings. As in the case of the macOS findings, I could add an override. As noted earlier, not every finding is just as it is presented by OpenVAS. Some of them may be false positives, for instance. You can change the severity of the finding to false positive by using an override. You can see the dialog box that lets you set override parameters, including setting the severity to false positive, in [Figure 5.17](#).

New Override

NVT Name: Mac OS X 10.5.2 Update / Mac OS X Security Update 2008-001

Active: ☒ yes, always
☐ yes, for the next 30 days
☐ no

Hosts: ☐ Any ☒ 192.168.86.170

Location: ☐ Any ☒ general/tcp

Severity: ☐ Any ☒ > 0.0

New Severity: ☒ False Positive ☐ Other:

Task: ☐ Any ☒ Regular%20Scan

Result: ☒ Any ☐ Only the selected one (33e7a228-8bd6-4028-990b-e363088beede)

Text:

Associated Result

Vulnerability	Severity	QoD	Host	Location	Actions
Mac OS X 10.5.2 Update / Mac OS X Security Update 2008-001	10.0 (High)	97%	192.168.86.170	general/tcp	

FIGURE 5.17 Setting an override

Of course, a false positive is not the only change to severity that you can make. You can either increase or decrease the severity. You may know quite a bit more than OpenVAS does, especially if you either work for the company you are testing for or are working closely with them rather than as a strict adversary. If there are mitigations already in place, you may find you want to lower the severity. There may also be reasons to increase the severity provided by OpenVAS. After all, this is a generic finding, sometimes provided by the software vendor.

You can also set other parameters in the override, if it's not a matter of severity that you want to change. If you were to override the severity, you probably want to add a note as well, explaining what the change was and why it was being made. This allows any others who have access to OpenVAS to review the changes that were made and know why the changes were made. In addition to making changes to parameters like severity, you can add notes to each of the findings.

Using notes and overrides allows you to use OpenVAS as more of a vulnerability management tool, to some degree, rather than just a vulnerability scanner. You can use it for historical purposes, to identify known vulnerabilities, or to make alterations to different parameters. This stored information may be useful because anyone tasked with protecting systems and networks will be constantly fighting against vulnerabilities. Historical information, especially mitigations and what was done to remediate the vulnerabilities, will be useful for security professionals.

Nessus

Nessus is the parent of OpenVAS, which makes it worth looking at, especially to see how it has diverged from the path OpenVAS took. While Nessus is a commercial product, there is a home license so you can use it on your home network to compare against OpenVAS and also see another approach to vulnerability scanning. When you log in, you're taken to your list of scans, which will be empty at first. To start a scan, you would click the New button, which will take you to a list of the different scan policies that are available. You can see some of the scan policies that are built into

Nessus in [Figure 5.18](#). You will see *Upgrade* printed across some of the scan policies. This means these scan policies are available only in the commercial version and not available in the Home license.

If we select the Basic Network Scan as our starting point, we can begin to select our target and then go through customization. [Figure 5.19](#) shows the configuration settings for the Basic Network Scan. You'll see in the first screen that you provide the name of the scan. Keep in mind that you are creating a configuration here, and you can run that configuration multiple times. Each run will have a time stamp associated with it, so you don't need to add your own date and time as part of the name. You'll want to name it something that is meaningful to you so you can differentiate this configuration from other configurations.

You might, for instance, have a configuration where you have a specific set of credentials. [Figure 5.20](#) shows the configuration for credentials. You'll see that you can create SSH and Windows credentials for host authentication. For each of these types, you will be able to create multiple configurations. You could use SSH key authentication for some systems and then also have an authentication setting for where you need to use the username and password. According to the interface, you can have unlimited credentials for both SSH and Windows. To create another set of credentials, there is a button at the end of the line indicating the credential type.

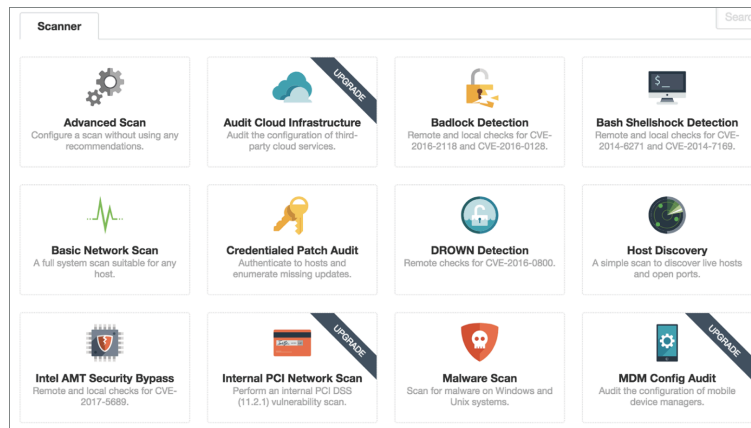


FIGURE 5.18 Scan policies in Nessus

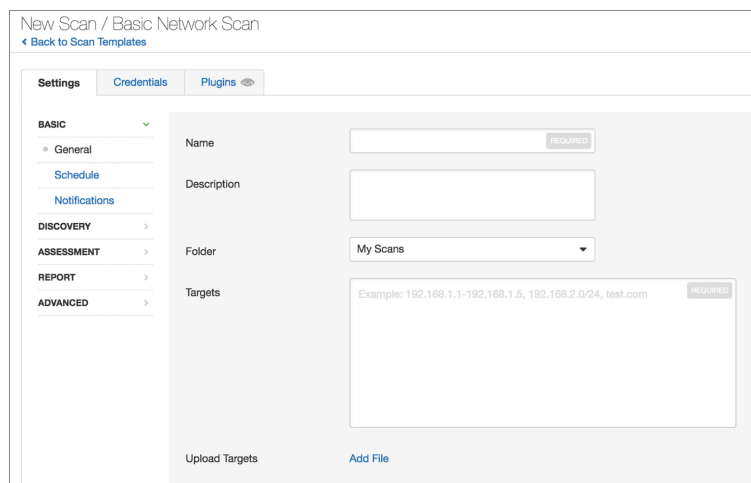


FIGURE 5.19 Scan configuration settings

FIGURE 5.20 Credentials configuration settings

In addition to host credentials, you can set authentication credentials for database, miscellaneous, and plain text. The last category is for protocols like HTTP, FTP, and SMTP, among others. Miscellaneous gives you settings for VMware and Palo Alto firewalls. This provides the means for Nessus to check local vulnerabilities across a variety of applications and also devices.

In addition to credentials, you can configure the plugins that you want to run. You can get to the plugins by clicking the Plugins tab across the top. Looking back at [Figure 5.19](#), you'll see a set of tabs vertically along the left side. This not only provides you with access to the basic settings that you can see, it also provides you with access to discovery, assessment, report, and advanced configurations. The Discovery tab lets you determine the type of discovery you will be doing on the network, meaning it lets you set port scan parameters. By default, the port scan will be run against common ports. You can select to let it run against all ports, which may mean you could find additional vulnerabilities or at least some ports you may not have expected.

The Assessment tab lets you set parameters on what Nessus should scan with respect to web vulnerabilities. By default, Nessus won't scan for web vulnerabilities. You can set different levels of web vulnerability scanning. There is another setting on the Assessment tab that is important. By default, Nessus will limit the number of false positives found. It should be noted that all the settings mentioned here are in the Basic Network Scan policy. Other scan policies will have different settings, some of which you will be able to change. To get access to all of the settings, you should go through the Advanced Scan, where all the settings are exposed and can be changed.

The Report tab lets you adjust the report verbosity. You may want to limit the number of details provided, keeping information in the report to a minimum. The Nessus reports, by default, are fairly comprehensive. Each finding will contain details about the vulnerability as well as its severity, references associated with the vulnerability, and potential remediations. Including enough detail so everything will make enough sense to people who may not be very familiar with Nessus can take up a lot of space.

As the scan is running and certainly when it is complete, you will be able to view the results. Initially, you will get a list of all the vulnerabilities when you open up the scan by clicking its name under My Scans. You can

see a partial list in [Figure 5.21](#). Alongside the list of vulnerabilities identified, you will see a chart of all the identified vulnerabilities to indicate which categories have the most vulnerabilities. In the case of the scan shown, the vast majority are informational messages, while the second highest number of vulnerabilities are in the medium severity category.

FIGURE 5.21 Scan results list

The list of hosts is ordered by the total number of issues identified by Nessus, though findings are not all vulnerabilities since there are informational items, which are not vulnerabilities. Nessus identified a total of 50 hosts on the network, and the host with the IP address ending in .52 had the highest number of vulnerabilities based on vulnerabilities Nessus knew about at the time of the scan. This is not to say there are no other vulnerabilities, which may not be known by Nessus. It's also possible for there to be false negatives, meaning Nessus didn't identify a known vulnerability in spite of the fact that the vulnerability existed on the scanned host. In the right column, you will see a percentage. This percentage indicates how complete Nessus thinks the scan against that host is. This snapshot was taken mid-scan.

Across the top of the page, you will see three tabs. The first one, and the one that is presented first, is the list of hosts. This is, as noted earlier, ordered by total number of vulnerabilities. The second tab is the number of vulnerabilities. This is ordered by the severity of the finding. The critical issues are on top, followed by the high, then the medium, and so on. A scan of hosts on the local network identified two different critical issues. One of them was related to software from the Mozilla Foundation. According to Nessus, either Thunderbird or Firefox is installed on the target host, though the version installed is no longer supported. The second issue has to do with a macOS system. The version of the operating system installed is one behind what is the most current. [Figure 5.22](#) shows details related to the Mozilla vulnerability.

FIGURE 5.22 Finding details

You will see in the output a description of the finding. You will also see the solution, which is to upgrade to a supported version. Below that there are references to other sites that can provide additional details about the issue. Below the reference sites are the details from the scan plugin. This shows the version of the application found as well as the current version of the application that is available. This shows that the version installed is five versions back, and the current version available and supported by the Mozilla Foundation at the time of the scan was 61.0. What you don't see in this output is the IP address of the second host where this problem was identified. Only one of the IP addresses is shown for brevity.

Also along the top is a tab labeled Remediations. This provides output indicating how some of the identified vulnerabilities can be remediated. You will see in [Figure 5.23](#) that the four remediations identified are all related to upgrading to the latest version of software, including, in one case, updating the version of the operating system in use.

FIGURE 5.23 Remediations list

As has been noted several times, severities will vary, and just because Nessus believes a severity should be at a certain level doesn't mean that you, with your knowledge of the environment you are working with, will agree. You can create rules in Nessus so findings associated with certain hosts may be automatically recategorized. This can be done from the Plugin Rules tab along the left navigation frame in the main view. [Figure 5.24](#) shows what the dialog box looks like for the Plugin Rules settings.

You can specify a host or just leave that field blank for all hosts. You would need to specify the plugin ID that the rule applies to. With all the parameters in place to identify what the rule applies to, you just set the severity you want associated, and Nessus will take care of the rest.

FIGURE 5.24 Plugins Rules settings

Unlike OpenVAS, Nessus doesn't give you a way to add notes to findings. One thing you do get, though, that you don't get in OpenVAS is an audit trail. You can search by plugin ID and by host and get additional details about the plugin that was run. As an example, searching for the plugin ID from the critical macOS out-of-date finding shows the IP addresses for hosts where Nessus couldn't identify the OS, hosts where Nessus did identify the OS but it wasn't macOS, and also IP addresses where the operating system was correct but the version number wasn't correct, meaning there was no finding resulting from the run of the plugin.

Like OpenVAS, Nessus uses Network Attack Scripting Language (NASL) scripts. They are stored with the rest of the Nessus installation. On Windows, the installation would be in the Program Files directory. On Linux, the files are stored in `/opt/nessus` with the plugins in `/opt/nessus/lib/plugins`. If you want to see exactly what a plugin does so you can verify the finding, you should check the script. You can use the plugin ID to identify the file that is run for that plugin. The scripts, once you get used to reading them, will provide you with the details on how to replicate the vulnerability test so you can verify. This verification is not only important to rule out false positives, but if it is actually a vulnerability, you will need to have documentation to present to the company or organization you are working with.

Vulnerability scanning is just a stage in the testing. It is not the end. Vulnerabilities should always be verified. Additionally, if you are expected to identify as many vulnerabilities as you can, you will need to move to exploiting these vulnerabilities in hopes of identifying more. While this may require the use of exploit tools like Metasploit or even tools that are custom-developed, it could be that you need to create a

packet that looks a particular way. No matter what the findings are, though, they need to be verified before being presented as findings.

Looking for Vulnerabilities with Metasploit

Metasploit is a versatile tool. Certainly, you can use it for exploiting applications as well as the port scanning we did earlier. As it's a framework and there are a lot of modules, it shouldn't come as a big surprise that there are lots of vulnerability scanners. One example is a scanner module for the Eternal Blue vulnerability. This is a vulnerability in the implementation of the Server Message Block (SMB) protocol that was discovered by the National Security Agency (NSA), which developed an exploit for it, which was released without the NSA's approval by a group called the Shadow Brokers. The following is one of the vulnerability scanners that looks for systems vulnerable to the Eternal Blue exploit. While the fix for this vulnerability has been out for years at this point, there are still many systems in production that are vulnerable to it.

```
msf6 auxiliary(scanner/portscan/syn)> use auxiliary/scanner/smb/smb_ms17_010
msf6 auxiliary(scanner/smb/smb_ms17_010)> set RHOSTS 192.168.4.0/24
RHOSTS => 192.168.4.0/24
msf6 auxiliary(scanner/smb/smb_ms17_010)> run

[-] 192.168.4.10:445 - Host does NOT appear vulnerable.
[-] 192.168.4.15:445 - Host does NOT appear vulnerable.
[*] 192.168.4.0/24:445 - Scanned 26 of 256 hosts (10% complete)
```

There are a large number of scanners available in Metasploit. Not all the scanners are directly related to the vulnerabilities. Searching for the word *scanner* in Metasploit returned 619 results, with some of the output following. In just the output fragment here, you can see that some of them are related to known vulnerabilities while some are looking for instances of applications that may have a particular configuration issue.

```
192 auxiliary/scanner/http/manageengine_deviceexpert_user_creds 2014-08-2
193 auxiliary/scanner/http/manageengine_securitymanager_traversal 2012-10-1
194 auxiliary/scanner/http/mediawiki_svg_fileaccess
195 auxiliary/scanner/http/meteocontrol_weblog_extractadmin
196 auxiliary/scanner/http/mod_negotiation_brute
197 auxiliary/scanner/http/mod_negotiation_scanner
198 auxiliary/scanner/http/ms09_020_webdav_unicode_bypass
199 auxiliary/scanner/http/ms15_034_http_sys_memory_dump
```

It's not the most comprehensive vulnerability scanner, but if you are already in Metasploit for other purposes, you can make use of vulnerability scan modules to look in a targeted way for some vulnerabilities. If you have a good way to detect a vulnerability and feel like writing a little Ruby, you can write a module for Metasploit that can look for that vulnerability.

Packet Crafting and Manipulation

When you are sending data out over the network, there is a clear path it follows before exiting the network interface on your system. We've gone over this, to a degree, by talking about the Open Systems Interconnection (OSI) model. Let's say that you are visiting a web page. You enter a URL into the address bar. Your browser takes the input and creates the HTTP request headers that are needed to send to the server. For simplicity, we'll skip the encryption pieces and just talk about how the complete packet is put together.

The application makes a request of the operating system to open a connection to the server. This triggers the operating system to build a packet using information provided by the application. This includes the host-name or IP address as well as the port number. Unless otherwise provided, the port number will default to either 80 or 443, depending on whether the communication is HTTPS or HTTP. This information will allow the operating system to create the necessary headers for both TCP and IP, layers 4 and 3.

All of this is to say that the application initiates requests based on interaction from the user. It follows a clear path, and the information placed into the necessary headers for each protocol is coherent and easily traced back to the original source of the information. Sometimes, though, you may need to send data that doesn't follow a coherent path. It could be that you need to manipulate headers with data that wouldn't normally be found in the header fields. Each header field is a known size and is binary, which means you aren't going to be sending a character instead of a number, for instance. Nothing in the network headers, looking at layers 4 and below, for sure, is data that would go through an ASCII decode to be converted to character data.

There are a number of tools that can be used to craft or otherwise manipulate the header data. Some of these are designed for the sole purpose of creating packets that would look the way you want them to look. This may be a tool like packETH, which uses a GUI to let you set the fields. Others have other purposes that allow you to interact with the target system in a way that you may not otherwise be able to do without writing your own program. A tool like `hping` will let you build a packet based on the command-line parameters. Using a tool like `hping`, you could assess the response from the system. Finally, you may want to mangle the packet using a set of rules, which would put the operating system's network stack to the test, to see if it can handle poorly constructed packets.

hping

The program `hping` is considered by the developer to be the Swiss Army knife of TCP/IP packets. You could use it as a straightforward ping program, sending ICMP echo requests. Since `hping` is primarily a packet crafting program, allowing you to initiate connections using different protocols with the header settings you want, the default mode may not work very well for you. By default, if you don't specify anything other than the target host or IP address, `hping` will send messages to port 0 on your tar-

get with a varying source address. Address 0 is essentially an invalid destination since it is considered reserved and has no purpose. You shouldn't get any response from the system you are sending traffic to. If you do, the target host is really violating the protocol. While `hping` uses TCP for this, port 0 is invalid for both UDP and TCP.

While you can use `hping` as a replacement for the ping program, by calling it with the `-1` parameter, meaning you are using ICMP mode, you can also create connections to specific ports. You will get the same behavior you would get with the ping program, meaning you will be getting the "aliveness" of the system and the round-trip time. You will get something even more detailed, though, since you will know whether a particular service is up and running. This may be useful if you are doing testing against an application. You may want to know when the service fails. You will get a lot of detail from the response in addition to the round-trip time. You can see in the following code listing a run of `hping3` against a web server on a target system.

SUPERUSER PRIVILEGES

Because `hping` is doing things the operating system wouldn't do, just like `nmap` earlier, we need to run it with root-level permissions. It is shown here as running as the root user. While it's bad to log in directly as root, you can spawn a root shell for a short period of time. Preferably, this should be done using `sudo`.

Sending SYN Messages to a Target System

```
kilroy@quiche:~$ sudo hping3 -S -p 80 192.168.86.1
HPING 192.168.86.1 (eth0 192.168.86.1): S set, 40 headers + 0 data bytes
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=0 win=29200 rtt=7.9 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=1 win=29200 rtt=7.9 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=2 win=29200 rtt=7.6 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=3 win=29200 rtt=7.5 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=4 win=29200 rtt=7.3 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=5 win=29200 rtt=3.0 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=6 win=29200 rtt=2.8 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=7 win=29200 rtt=2.7 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=8 win=29200 rtt=2.5 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=9 win=29200 rtt=2.4 ms
len=46 ip=192.168.86.1 ttl=64 DF id=0 sport=80 flags=SA seq=10 win=29200 rtt=2.2 ms
```

`hping` will provide you with all of the flags that are set in the response. This includes the SYN and ACK flags as well as the don't fragment bit, indicated by the DF in the response. You can see a lot of other details as well, including the IP identification number, the relative sequence number, and the window size as provided by the target host. In the case of a port where there is no listener, you won't get a message indicating that the host is unreachable or that the message timed out, as you would with a standard ping program. Instead, you will get the same response that you got from an open port. Instead of showing SA for flags, meaning that the

SYN and ACK flags were set, you will see RA, meaning the RST and ACK flags. The remote system reset the port, telling us that there is no application there. You will still get all of the other information, including the round-trip time, which will tell you how quick the target system is to respond to these messages, which will be a factor of network and operating system responsiveness.

HPING WITH BAD FLAGS SET

Raw sockets provide programmers with the ability to bypass the network stack. When a programmer uses raw sockets, the program is expected to handle all the things the network stack does, meaning all the values in the headers should be set. Raw sockets provide the programmer with complete control over what the packet will end up looking like. None of it has to be considered legal from the standpoint of the protocols, if you aren't expecting responses. This is not to say that you will always get a response, though. You could completely mangle the message to the target host.

As one example, take a look at the following command line. The offset for the TCP headers is being set incorrectly, which means the target network stack is being pointed to the wrong place.

```
root@quiche:~# hping3 -O 8 -s 15 -F -S -P -A -t 3 -p 80 192.168.86.1
HPING 192.168.86.1 (eth0 192.168.86.1): SAFP set, 40 headers + 0 data bytes
^C
--- 192.168.86.1 hping statistic ---
19 packets transmitted, 0 packets received, 100% packet loss
round-trip min/avg/max = 0.0/0.0/0.0 ms
```

Also, the SYN and FIN flags are both set, as well as the ACK and PSF flags. This is a flag combination that makes no sense. The source port is being set to 15, which would require administrative privileges to do, as do most things in `hping`, considering it is generally using something called *raw sockets*.

In addition to ICMP and TCP, you can send UDP messages. There are fewer parameters used to send UDP messages because of the limited number of options available in the UDP headers. We can, though, use `hping` to perform a port scan. We can scan a range of UDP ports by using the `--scan` (or `-8`) parameter. You can see this done in the following code. Using `--scan`, we need to specify the ports being targeted. This scan targets the administrative ports 1–1023. There are no ports listening on this host in that range. What was truncated from the output was all of the port numbers and associated service names that were found not to be listening. One other feature of `hping` is the ability to spoof addresses. Using the `-a` parameter, followed by an IP address, will have `hping` change the source address in messages going out. This will mean that you won't get any responses, because responses will be sent to the source address you specify.

UDP Port Scan with hping

```

root@quiche:~# hping3 --scan 1-1023 -a 10.15.24.5 -2 192.168.86.1
Scanning 192.168.86.1 (192.168.86.1), port 1-1024
1024 ports to scan, use -V to see all the replies
+---+-----+-----+---+-----+-----+
|port| serv name | flags |ttl| id  | win | len |
+---+-----+-----+---+-----+-----+
All replies received. Done.
Not responding ports: (1 tcpmux) (2 nbp) (3 ) (4 echo) (5 ) (6 zip) (7 echo) (8 ) (9 disc

```

One other feature worth mentioning is the ability to send packets of any size you want. To change the size of the data being sent, you would use `-d` followed by a byte count. This sets the body size of the packet in the headers. You can also fill the packets by specifying a filename using the `--file` parameter. This will read the contents of the file and use them to fill the data portion of the packet. You may be able to crash the application because the data being sent could violate protocol specifications.

packETH

Where `hping` uses command-line parameters, `packETH` takes a GUI approach to being able to set all the parameters. The sets of headers vary, depending on the protocols selected, and each of the lower-layer headers indicate the next protocol, meaning the next set of headers. When you select which protocols you are using, `packETH` will adjust to provide all of the header fields for the protocol you have selected. [Figure 5.25](#) shows the IP header fields as well as the TCP header fields. You will also see where IP is selected as the next protocol in the layer 2 header. You can't see the layer 2 header in this screen capture, but you would be able to set addresses, determine what version of the layer 2 protocol you are using, and also add in 802.1q fields, which provides a tag field to indicate which virtual LAN (VLAN) the frame should be on.

In addition to setting headers, you can add your own data that would go into the payload. You don't need to have data to include, though, if you would prefer to just have data filled in to a certain size. [Figure 5.26](#) shows the TCP headers filled in with the data payload also filled in. On the right side of the screen capture, you can see two edit boxes. One of them is the data pattern, which is expected to be in hexadecimal. The other one is the number of instances of the pattern provided. The first field is set to `ab`, and the number of iterations is set to 500. Once you have the pattern and number, you apply the pattern and your data payload will be filled in. You'll notice that it is formatted just as you'd expect a hexadecimal dump to be formatted, with each hexadecimal byte separated from the others.

FIGURE 5.25 packETH interface

FIGURE 5.26 Data pattern fill

While you can create packets following known and understood fields, you can also create your own packets. Your layer 2 headers have to be set with MAC addresses in the source and destination so there is somewhere for the frame to go, but beyond that, you can do whatever you like by selecting User defined payload in figure. This would leave out all layer 3 information and include only what you wanted to include, whether it's text or a hexadecimal fill pattern. [Figure 5.27](#) shows a payload created using a pattern. Using text caused an error with the next-layer protocol specified because it's not set up to take raw text. You need to create a hexadecimal pattern instead. You'll see at the bottom of the screen capture that 60 bytes were sent, which includes the network layer payload we specified.

FIGURE 5.27 Network layer data fill

Once you have your packet built, you can send it. Clicking the Send button in the toolbar will send a single packet. If you want to send more than a single packet, you will have to use either the Gen-b or Gen-s button. Gen-b gives you the ability to specify the number of packets to send. You'll be able to indicate the bandwidth you want to use, or you could also indicate the inter-packet gap, which is the delay between packets being sent. Gen-s gives you the ability to generate streams. A stream can be a defined pattern of packets that have been saved to different files. Once you have the pattern defined by indicating the packets you want to use, you can tell packETH how you want to send them—burst, continuous, or random. You can also indicate the total number of packets you want to send as well as the delay.

Speaking of loading packets from a file, you can save the packets you create. This allows you to create a number of packets and load them into Gen-s mode, but it also allows you to create a packet and load it up to send it anytime you want without having to re-create the packet. If you want to use an existing packet you have captured as your starting point, you can also load a packet capture (PCAP) file. When you select a frame from the list in the PCAP view, that frame will show up in the Builder view.

fragroute

`fragroute` is a program used to mangle packets before they are sent to a target you specify. It works by making adjustments to the routing table so all messages going to the target are sent through the `fragroute` application first. To make `fragroute` work, you need to create a configuration file. This configuration file has directives telling `fragroute` how to handle packets that pass through the application. In the following code listing, you can see a configuration file with a handful of directives that are guaranteed to create really messed-up network traffic.

fragroute Configuration File

```
kilroy@lolagranola $ cat frag.conf
delay random 1
dup last 30%
ip_chaff dup
ip_frag 128 new
tcp_chaff null 16
order random
print
```

The directives here tell `fragroute` to do a number of things to packets. The first thing is to delay random packets by 1 millisecond. Next, there is a 30 percent chance of duplicating the last packet. The `ip_chaff` line adds duplicate packets into the queue. When messages are sent out, they have a maximum transmission unit (MTU) size that is dictated by the data link protocol. With Ethernet, that is 1,500 bytes, though it's possible to get something called *jumbo frames* that are much larger. More commonly, you will see an MTU of 1,500. Any message that is larger than the MTU gets fragmented. Using `fragroute`, though, we are going to force the packets to get fragmented before sending. That happens in the `ip_frag 128 new` line. We will be fragmenting at 128 bytes, which is enough for header data and a little bit more. Anything large being sent, such as an image file, will have a large number of fragments.

The line starting with `tcp_chaff` does the same thing that `ip_chaff` does, working instead on the Transport layer. The TCP segments being inserted will have null TCP flags, as specified in the configuration. We could also have had invalid checksums, older time stamps, or other bogus information in the TCP header. This would have caused these messages to be rejected on the far end of the conversation, after inspection. Finally, the order of the messages will be randomized, so they will be out of order and need reassembly on the far end, and then message details will be printed.

Using a simpler configuration file, you can see a run of `fragroute` in the following code listing. This is the configuration file that was installed by default with the `fragroute` package on a Kali Linux system. This configuration file uses `tcp_seg` to break up TCP data into specified segment sizes. After that, it uses `ip_frag` and `ip_chaff` as mentioned earlier. Then, it will set an order and print the message details, which you can see.

fragroute Run Against Target

```
root@quiche:~# fragroute -f /etc/fragroute.conf 184.159.210.190
fragroute: tcp_seg -> ip_frag -> ip_chaff -> order -> print
192.168.86.57.18294> 184.159.210.190.17766: SR 1400140884:1400140908(24) ack 1802781559 v
192.168.86.57.43460> 184.159.210.190.4433: S 2873730507:2873730507(0) win 29200 <mss 1460
192.168.86.57.21314> 184.159.210.190.29050: S 810642531:810642543(12) ack 1802326352 win
192.168.86.57.43460> 184.159.210.190.4433: S 2873730507:2873730507(0) win 29200 <mss 1460
192.168.86.57.19306> 184.159.210.190.22387: R 1297315948:1297315960(12) ack 2020107846 wi
192.168.86.57.43460> 184.159.210.190.4433: S 2873730507:2873730507(0) win 29200 <mss 1460
```

```
192.168.86.57.26963> 184.159.210.190.21350: SFP 1950696520:1950696548(28) win 27988 urg 2
```

What you will likely notice when you use `fragroute` is that what you are trying to do to interact with the target will fail. In the preceding example, I used `openssl s_client` to initiate a connection with the web server using SSL/TLS. The connection never completed, presumably because the packets getting to the target were so mangled and out of order that the network stack didn't know what to make of them. The point of running `fragroute`, though, isn't necessarily to make the connection. Sometimes, the point is just to see if you can make the network stack on the target system fail, which may take the kernel with it, causing the entire system to be unavailable and forcing a reboot or restart.

Evasion Techniques

Any target organization you test against will have security mechanisms in place to defend itself. This may be firewalls or intrusion detection systems. It may also have intrusion prevention systems. Any of these could thwart your efforts either by blocking them or by issuing an alert, which may result in the discovery of your actions. Either of these would be bad things. Fortunately, there are some evasion techniques that may help you get around these devices so you can keep plugging along. Some of the tools we have already looked at will help you with these evasive procedures. The common evasion techniques are as follows:

- **Hide/Obscure the Data** You could use encryption or obfuscation to disguise what you are doing. Encrypted traffic can't be investigated without violating the end-to-end nature of encryption. The goal with encryption is that the message is encrypted from the sender to the recipient, without being decrypted at waypoints in between. You could also encode the data using various encoding techniques, including URL encoding, which replaces characters with the hexadecimal value of their ASCII code.
- **Alterations** Intrusion detection/protection systems in particular will often use something called a *signature*. In the case of malware, this may be a cryptographic hash value that can be compared against a database of known malware. If there is a match of the hash, the messages can get dropped. When it comes to a cryptographic hash, though, the change of a single character in the file contents will yield a completely different hash value, meaning whatever you are doing won't get detected. This strategy is commonly called polymorphisms, from *polymorph*, meaning many shapes or forms.
- **Fragmentation** Fragmentation attacks can be used to evade network security mechanisms simply because these devices, when they are in-line, would take time to reassemble the messages before the adversarial activity would be seen. This reassembly takes time and so some devices just don't bother because the reassembly and detection can add latency to communications. This depends on the device and decisions made by the developers of the device. You can use a tool like `fragroute` to help you with the fragmentation.

- **Overlaps** When messages are fragmented, it may happen at either the Network layer or the Transport layer, as you saw from looking at `fragroute`. When the messages need to be reassembled, all of the pieces need to be there and in a sane state so the puzzle can be fit back together. When using TCP, you can overlap sequence numbers. This is essentially the byte count that has been sent. You may send two TCP segments that appear to occupy the same space in the puzzle being put back together. The IDS and the target OS may decide to put the puzzle back together differently. This may happen if one decides to favor the newer information while the other favors the older. The OS needs to decide whether the first message received was valid or the last message received was more valid.
- **Malformed Data** Protocols are sets of rules about how communications are expected to happen. If you violate those rules, you can get unexpected results. Even if you aren't violating the rules but instead are taking advantage of loopholes, you can get some useful data. This is why `nmap` uses Xmas, FIN, and NUL scans. The behavior is unexpected, though not technically illegal from the standpoint of the protocol. Similarly, there are details in the protocols that may be handled differently across different network stacks. The URG pointer may be handled differently across different operating systems. This could be used to get around an IDS and still have the target system respond the way you want.
- **Low and Slow** Fast scans can be easy to detect. Harder to detect are scans that are taking place over a long time frame. Imagine a single scan packet being sent once an hour. This can be very time-consuming to perform, but when you are talking about individual messages, it's far less likely that the IDS or firewall would identify them as a port scan. Taking your time can be beneficial. You could use the `nmap` throttling parameter to really slow your scans down.
- **Resource Consumption** It may be possible to get devices to fail open by consuming resources such as CPU or memory. If you can exhaust either of these resources, it may be possible to get subsequent messages to just pass through once the device has failed.
- **Screen Blindness** In the case of IDS, the device or software will issue alerts. It is expected there will be someone looking at those alerts. If you can generate enormous volumes of alerts from traffic you don't care about, you can cause the people looking at the alerts to go screen blind, meaning they just aren't seeing the important details anymore because they are overwhelmed by what they are looking for. This way, you can set up a smoke screen with a lot of bogus alert traffic and then send your real data through the screen.
- **Tunneling** A tunnel is a way of transmitting data inside something else. For example, the Generic Routing Encapsulation (GRE) protocol can create a tunnel by taking packets and encapsulating them inside GRE packets. This makes it look like what is passing through is a GRE packet when there is really something in the payload. GRE is a protocol that has been designed to tunnel traffic in cases where you want to handle the routing on the receiving end rather than the sending end. Other protocols have been used for tunneling attacks, including SSH, HTTP, ICMP, and DNS. These tunneled attacks require software on the

receiving end that can extract the tunneled messages and place them on the target network.

Keep in mind that with devices like stateful firewalls, once you get the first message through, subsequent messages may be allowed by default because they are part of an established connection. Of course, these techniques for evasion have all been around for a very long time, so it's possible that the firewall or IDS vendor knows how to detect most of them. This means you may not be able to make the evasions work. It is still worth knowing about them and trying them to see if you can get through.

There are some that are more likely to work than others, because they don't rely on the firewall or IDS vendor. Encryption, for instance, should always work because the firewall and IDS simply can't see the data since the keys are negotiated by the two endpoints. Encryption, though, requires a service on the receiving end that understands encryption and can negotiate with the sending application. An evasion technique that doesn't rely on technical means is overwhelming the person looking at the alerts. This does, though, assume that there isn't technology in place to weed out the extraneous messages, only presenting interesting alerts to the operator.

Evasion with *nmap*

`nmap` has its own evasions built in, to a degree. There are the ones that are mentioned earlier, with what are essentially malformed requests. These are packet configurations that simply shouldn't exist in the real world. The expectation is that using these packet configurations, the network security in place may simply ignore them. Unfortunately, these are so well-known that firewalls and intrusion detection systems are more than capable of seeing these messages. One that may be harder to see is the idle scan, which makes use of a system that isn't communicating on the network. This allows `nmap` to calculate the correct IP identification number that should be received based on the system having to respond with reset messages to packets from the target.

The way the idle scan works is the system performing the scan spoofs messages, making it look like the messages are coming from an idle system. The receiving system then sends all responses to the idle system, while the scanning system periodically sends a message to the idle system to get the current IP identification value. Based on this, the sending system can determine what packets may have been received and sent since the sending system knows what it sent out.

While the scan can be easily seen by firewalls or detection systems, what won't be seen is where the scan originated from. If the scanner uses a victim, or idle, system that has nothing to do with the scanner, it won't be clear who is actually performing the scan, even if the fact of the scan is picked up. The person, say you for instance, doing the scan will be hidden because the scan is blind.

Another way to "blind" the defender is to use a number of decoys. You can have `nmap` create a diversion by generating a lot of bogus traffic

alongside the port scan you actually care about. You do this by passing the `-D` flag along with the decoy parameters. While you can specify addresses to use, it's just as easy to let `nmap` randomize the source addresses. This is how you would let `nmap` create a random diversion using three random source addresses.

```
$ sudo nmap -D rnd:3 192.168.178.1
Starting Nmap 7.93 ( https://nmap.org ) at 2022-11-20 19:19 CET
Nmap scan report for unnamed (192.168.178.1)
Host is up (0.013s latency).
Not shown: 993 closed tcp ports (reset)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
443/tcp   open  https
554/tcp   open  rtsp
5060/tcp  open  sip
8089/tcp  open  unknown
8181/tcp  open  intermapper

Nmap done: 1 IP address (1 host up) scanned in 1.54 seconds
```

This looks like a normal `nmap` scan, until you take a look at the packet capture. The packets shown following were captured during that scan. This scan was done from inside a virtual machine, so the actual source address would have been 10.211.55.11. You can see three different source addresses on top of the actual source. The three addresses are 212.236.76.245, 53.129.176.4, and 41.97.136.192.

```
19:19:25.239139 IP 212.236.76.245.47924> 192.168.178.1.3006: Flags [S], seq 3917877688, wi
19:19:25.239141 IP 10.211.55.11.47924> 192.168.178.1.3006: Flags [S], seq 3917877688, win
19:19:25.239147 IP 53.129.176.4.47924> 192.168.178.1.3006: Flags [S], seq 3917877688, win
19:19:25.239149 IP 41.97.136.192.47924> 192.168.178.1.3006: Flags [S], seq 3917877688, wir
19:19:25.239152 IP 212.236.76.245.47924> 192.168.178.1.65129: Flags [S], seq 3917877688, v
```

On top of creating decoys, as discussed previously, you can have `nmap` take care of fragmentation for you. While `fragroute` is good and can be used across many different circumstances, having `nmap` take care of fragmentation is easy, if all you need to do is scan. This is an `nmap` scan with fragmentation.

```
$ sudo nmap -f -p80,443 192.168.178.1
Starting Nmap 7.93 ( https://nmap.org ) at 2022-11-20 19:26 CET
Nmap scan report for fritz.box (192.168.178.1)
Host is up (0.12s latency).

PORT      STATE SERVICE
80/tcp    filtered http
443/tcp   filtered https
```

Another way of accomplishing something similar is to have `nmap` set the MTU. This is a field in the IP header that tells the network equipment how large the IP packets can be based on the transmission medium. Some networks will only accept data frames of a limited size. Ethernet, for exam-

ple, allows a maximum transmission unit of 1,500 bytes. Any data segment larger than that needs to be fragmented, with the fragmentation headers set appropriately. It is up to the receiving party to put the complete message back together in the right order. If you set the MTU small enough, some intermediate devices like routers or firewalls may (note this is a may, not a will or even a should, as this is not the 1990s any longer with limited processing or storage capabilities) opt to send the messages on rather than reconstructing to review. Letting `nmap` set the MTU looks like this, keeping in mind that the MTU needs to be a multiple of 8:

```
$ sudo nmap -f -p80,443 192.168.178.1
Starting Nmap 7.93 ( https://nmap.org ) at 2022-11-20 19:26 CET
Nmap scan report for fritz.box (192.168.178.1)
Host is up (0.12s latency).

PORT      STATE      SERVICE
80/tcp    filtered  http
443/tcp   filtered  https
```

Additional techniques in `nmap` for evading firewalls and intrusion detection systems include spoofing the originating MAC address, which is a limited-use scan. It's used to take advantage of a trust relationship in the firewall on the local network, trusting a specific MAC address. If your target is remote, this type of scan simply won't work. The first time your frame hits a network device, like a router or firewall, the frame's header, including your MAC address, will be removed, only to be replaced by the appropriate MAC address on the other side of the firewall or router. If you want to try it, though, you can use `--spoof-mac` followed either by a MAC address or by a vendor. If you specify a vendor, `nmap` will use the appropriate OUI, along with a random three octets to identify the individual host.

Finally, you can spoof the source port. Some firewalls look at the source port because it's necessary for some applications, like DNS or FTP. While this is largely not a problem any longer, some firewall administrators, especially if the firewall implementation is getting a little long in the tooth, will check the source port number and allow traffic based on that. You could use `nmap -g 53`, for example, if you wanted `nmap` to send traffic from a source port of 53, suggesting it was DNS-related.

Protecting and Detecting

All of this is on the offensive side, where you will be looking for footholds or pieces of information you can use later. One thing to keep in mind when it comes to working with a client is your job is not just to break into systems and find weaknesses. You will be expected to provide recommendations on how to keep other people from following the paths that you used to get into systems. This means you not only need to know how to break in, but you also need to know how to keep yourself from breaking in.

Fortunately, when it comes to scanning, it's not that hard to protect against or at least detect activities like port scans. They are noisy, even if you are using a low and slow approach with long waits between packets going out. Sending requests to 1,000 ports over any period of time is going to be a little strange. While `nmap` refers to them as well-known ports, that's a comparative term. In reality, there are a handful of ports, considerably less than 1,000, that are in common use. Let's say that there are 20 ports, for the sake of the argument, that you would expect to see on your network, given everything that happens there. That leaves 980 ports that you should never see messages for on your network. Anytime you see any of those 980 ports getting requests, much less all of them, something is probably amiss. You can block all of that traffic, but you can also detect it. This may be done with either the firewall or a network intrusion detection system.

Vulnerability scanners are similarly noisy. Even in cases where the scanner knows the ports/applications ahead of time, there are a lot of requests that are going out. Almost all of them are going to look bad. This is where an intrusion detection system can help out. You may not be able to, or even want to, block traffic from a vulnerability scanner because, in reality, it will be or at least look like a legitimate request. You may be able to notice, though, that the requests match known vulnerabilities. Network intrusion detection systems should be able to detect this type of activity.

When it comes to attacks like packet crafting or spoofing attacks, there are ways to protect against these as well. Perimeter firewalls should be blocking traffic that should never happen. This includes private addresses from the RFC 1918 space, which are not, by convention, routable over the Internet. If an address can't come in over the Internet, anything with one of those addresses that comes in from the Internet can't be legitimate. Similarly, a spoofing attack, say from an idle scan, may appear to come from an inside host. Any address that lives on the inside of the network can't possibly originate from the outside, so those should be blocked as well.

Summary

Scanning will provide you with a wealth of information that will be necessary as you move forward with your testing and evaluation. There are different types of scanning, however. As you are scanning, you will want to identify ports that are open. The purpose of identifying open ports isn't just to get a list of ports. Ultimately, you want to identify the services or applications that are listening on the open ports. To identify these open ports, you would use a port scanner. The most commonly used port scanner is `nmap`, which can be used for more than just identifying open ports. It can also be used to identify application versions by grabbing banners. Also, `nmap` can identify the operating system running on the target.

While there are other port scanners available, including `masscan`, which is used for high-speed scanning, `nmap` is the only port scanner that has a scripting engine built into it. The scripting engine for `nmap` is based on the programming language Lua, but `nmap` provides libraries that will

give you easy access to the information `nmap` has so you can write scripts to better identify services and also perform tests such as identifying vulnerabilities. When you write scripts for the `nmap` scripting engine (NSE), you register ports with `nmap` so `nmap` knows to call your script when it finds the registered port to be open.

While `nmap` is commonly a command-line program, there is also a GUI that acts as a front end for `nmap`. Zenmap is a program that will call `nmap` based on a command specified, but it will also parse the results, providing them to you in different ways. You can look at all the services that were identified. You can also get a look at a topology of the network based on what `nmap` finds. While you can provide the same command to Zenmap as you do to `nmap`, Zenmap will also provide some scan types that you can run, like an intense scan. Selecting the scan type will fill in the needed command-line parameters.

Vulnerability scanners will not only look for vulnerabilities, they will also generally perform port scanning as part of looking for and identifying open ports. There are a number of vulnerability scanners available commercially. Very few vulnerability scanners exist that are open source. One, based on one that is now commercial, is OpenVAS. OpenVAS was forked from the last open source version of Nessus. One of the challenges of vulnerability scanners is the vast amount of work it takes to maintain them and keep them up-to-date, which is perhaps a primary reason why there are very few open source scanners.

Vulnerability scanners, like OpenVAS and Nessus, use plugins to perform tests. They probe the targeted host to observe behavior on the host to identify potential vulnerabilities. Not all identified vulnerabilities are real, however. Vulnerabilities that are identified by scanners but aren't real are called *false positives*. A false negative would be a vulnerability that did exist but wasn't identified. Vulnerability scanners are far from infallible because of the way they work. It may require manual work to validate the findings from a vulnerability scanner.

To test vulnerabilities and also perform scans, you may need to do something other than relying on the operating system to build your packets for you. There are multiple tools that you can use to craft packets, such as `hping`. This is a tool that can be used for scanning but also can be used to create packets using command-line switches. If you would prefer not to use the command line, you can use a tool like `packETH`. `packETH` presents you with all the headers at layers 2 through 4. You can also create a payload to go in the packet. `packETH` will also let you extract packets from a PCAP and then make changes to it. You can send individual packets to a target—either layer 2 or layer 3—or you could send streams. Using these crafted packets, you can get responses from your target that may provide you with necessary information.

Your target networks will likely have firewalls and an IDS installed. You will probably want to use techniques to evade those devices since they will likely prevent you from doing your job. There are multiple ways to evade security technologies, including encryption/encoding, causing the operator to go screen blind, or sending malformed messages to the target.

The best way to protect against these scans, whether they are port scans or vulnerability scans, is to use firewalls on the ingress of networks, which includes network segments like a virtual local area network. You may not always be able to protect against, meaning block, all of these tactics. This is where an IDS comes in handy, to let someone know something bad is happening.

Review Questions

You can find the answers in the appendix.

1. If you receive a RST packet back from a target host, what do you know about your target?
 - A. The target is using UDP rather than TCP.
 - B. The destination port is open on the target host.
 - C. The source port in the RST message is closed.
 - D. The target expects the PSH flag to be set.
2. What is the difference between a SYN scan and a full connect scan?
 - A. A SYN scan and a full connect scan are the same.
 - B. A full connect scan sends an ACK message first.
 - C. A SYN scan uses the PSH flag with the SYN flag.
 - D. The SYN scan doesn't complete the three-way handshake.
3. What is one reason a UDP scan may take longer than a TCP scan of the same host?
 - A. UDP will retransmit more.
 - B. UDP has more ports to scan.
 - C. UDP is a slower protocol.
 - D. UDP requires more messages to set up.
4. Why does an ACK scan not indicate clearly that ports are open?
 - A. The scanner has to guess.
 - B. ACK is not a supported flag.
 - C. The target system ignores the message.
 - D. ACK scans cause a lot of retransmits.
5. What is one reason for using a scan like an ACK scan?
 - A. It may get through firewalls and IDS devices.
 - B. It is better supported.
 - C. The code in `nmap` is more robust.
 - D. An ACK scan is needed for scripting support.
6. What does `nmap` look at for fingerprinting an operating system?
 - A. The operating system headers
 - B. The application version
 - C. The response from connecting to port 0
 - D. The IP ID field and the initial sequence number
7. What is `nmap` looking at when it conducts a version scan?
 - A. TCP and IP headers
 - B. Application banners
 - C. Operating system kernel
 - D. IP ID and TCP sequence number fields
8. What is an advantage of using `masscan` over `nmap`?
 - A. `masscan` has been around longer.
 - B. `nmap` is hard to use.
 - C. `masscan` can scan more addresses faster.

- D. `masscan` has access to scan more of the Internet.
9. If you were to see `hping -S -p 25 10.5.16.2`, what would you assume?
- A. Someone was trying to probe the web port of the target.
 - B. Someone was trying to probe an email port on the target.
 - C. Someone was trying to identify if SNMP was supported on 10.5.16.2.
 - D. Someone had mistyped `ping`.
10. If you were to see that someone was using OpenVAS, followed by Nessus, what might you assume?
- A. They were trying to break into a system.
 - B. They didn't know how to use Nessus.
 - C. They didn't know how to use OpenVAS.
 - D. They were trying to reduce false positives.
11. What is the difference between a false positive and a false negative?
- A. A false positive indicates a finding that doesn't exist, while a false negative doesn't indicate a finding that does exist.
 - B. A false positive indicates a finding that does exist, while a false negative doesn't indicate a finding that doesn't exist.
 - C. A false positive doesn't indicate a finding that does exist, while a false negative does indicate a finding that doesn't exist.
 - D. A false negative does indicate a finding that doesn't exist, while a false positive doesn't indicate a finding that does exist.
12. What would be the purpose of running a ping sweep?
- A. You want to identify responsive hosts without a port scan.
 - B. You want to use something that is light on network traffic.
 - C. You want to use a protocol that may be allowed through the firewall.
 - D. All of the above.
13. Which of these may be considered worst practice when it comes to vulnerability scans?
- A. Scanning production servers
 - B. Notifying operations staff ahead of time
 - C. Taking no action on the results
 - D. Using limited details in your scan reports
14. Which of these may be considered an evasive technique?
- A. Scanning nonstandard ports
 - B. Encoding data
 - C. Using a proxy server
 - D. Using `nmap` in blind mode
15. If you were to notice operating system commands inside a DNS request while looking at a packet capture, what might you be looking at?
- A. Tunneling attack
 - B. DNS amplification
 - C. DNS recursion
 - D. XML entity injection
16. What is an Xmas scan?
- A. TCP scan with SYN/ACK/FIN set
 - B. UDP scan with FIN/PSH set
 - C. TCP scan with FIN/PSH/URG set
 - D. UDP scan with SYN/URG/FIN set
17. What would you use MegaPing for?

- A. Running exploits
 - B. Running a port scan
 - C. Issuing manual web requests
 - D. Crafting packets
18. What would be a reason to use the Override feature in OpenVAS?
- A. You want to run a different plugin for a vulnerability.
 - B. You want to change the scanner settings.
 - C. You want to use TCP rather than UDP.
 - D. You want to change a severity rating on a finding.
19. What would you use credentials for in a vulnerability scanner?
- A. Better reliability in network findings
 - B. Authenticating through VPNs for scans
 - C. Scanning for local vulnerabilities
 - D. Running an Active Directory scan
20. What is `fragroute` primarily used for?
- A. Altering network routes
 - B. Capturing fragmented packets
 - C. Fragmenting application traffic
 - D. Fragmenting layer 2 and layer 3 headers
21. If you wanted to have `nmap` perform fragmentation for you, what command-line parameters could you use?
- A. `-f` and `--mtu`
 - B. `-f` and `-g`
 - C. `--mtu` and `-g`
 - D. `-g` and `-spooof-mac`
22. What type of scan would you use to take advantage of firewall rules that may be in place to accommodate protocols like DNS and FTP?
- A. Fragmentation
 - B. MAC spoofing
 - C. Source port spoofing
 - D. Idle scanning
23. When could you use MAC spoofing with your `nmap` scan?
- A. If your target is running DNS
 - B. If you ran a fragmentation attack at the same time
 - C. If you were remote
 - D. If you were on the local network
24. What number does the MTU setting need to be a multiple of when you are using MTU fragmenting with `nmap`?
- A. 2
 - B. 4
 - C. 6
 - D. 8
25. What command-line parameter would you use to perform a decoy scan with `nmap`?
- A. `-C`
 - B. `-D`
 - C. `-S`
 - D. `-F`

